

Neural Networks

EMEN 7029 & EMEN 7104 - Special Topic In
Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Admin	2
Review	4
Multivariate Linear Regression	17
Limitations of Linear Regression	21
History of Neural Networks	35
Biological Neurons	42
Artificial Neurons	63
Wide Neural Networks	80
Deep Neural Networks	84
Practical Considerations	90

Admin

Admin

How is assignment 1?

- Due next week, 42/63 submitted for EME, and 50/70 submitted for RAS
 - You are very disciplined, great job!
 - Make sure you submitted instead of drafted

Review

Review

We focused on how education affects life expectancy

Studies show a causal effect of education on health

- *The causal effects of education on health outcomes in the UK Biobank.*
Davies et al. *Nature Human Behaviour*.
- By staying in school, you are likely to live longer
- Being rich also helps, but education alone has a **causal** relationship with life expectancy

Review

Goal: Given your education, predict your life expectancy

$X = \mathbb{R}_+$: Years in school

$Y = \mathbb{R}_+$: Age of death

$\Theta = \mathbb{R}^2$: Parameters

$$f : X \times \Theta \mapsto Y$$

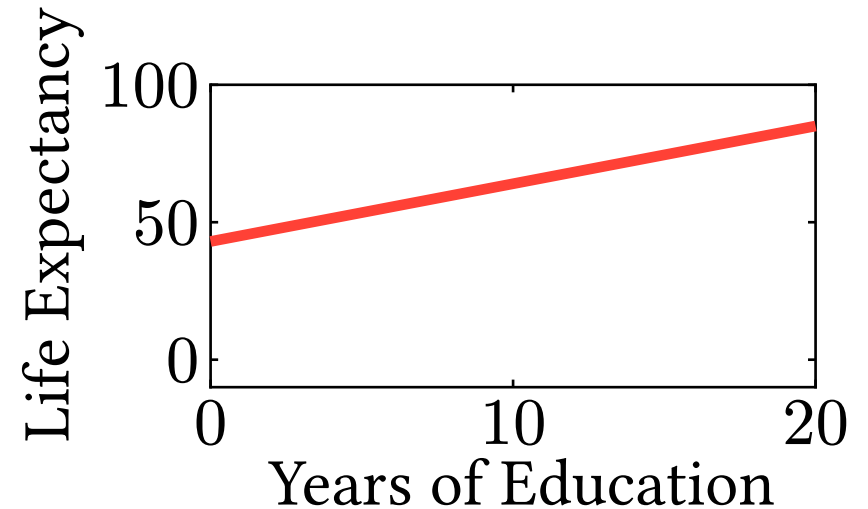
Approach: Learn the parameters θ such that

$$f(x, \theta) = y; \quad x \in X, y \in Y$$

Review

Started with a linear function f

$$f(x, \boldsymbol{\theta}) = f\left(x, \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}\right) = \theta_0 + \theta_1 x$$



Then, we derived the square error function

$$\text{error}(f(x, \boldsymbol{\theta}), y) = (f(x, \boldsymbol{\theta}) - y)^2$$

Review

We wrote the loss function for a single datapoint $x_{[i]}, y_{[i]}$ using the square error

$$\mathcal{L}(x_{[i]}, y_{[i]}, \theta) = \text{error}(f(x_{[i]}, \theta), y_{[i]}) = (f(x_{[i]}, \theta) - y_{[i]})^2$$

But we wanted to learn a model over **all** the data, not a single datapoint

We wanted to make **new** predictions, to **generalize**

$$\mathbf{x} = [x_{[1]} \ x_{[2]} \ \dots \ x_{[n]}]^\top, \mathbf{y} = [y_{[1]} \ y_{[2]} \ \dots \ y_{[n]}]^\top$$

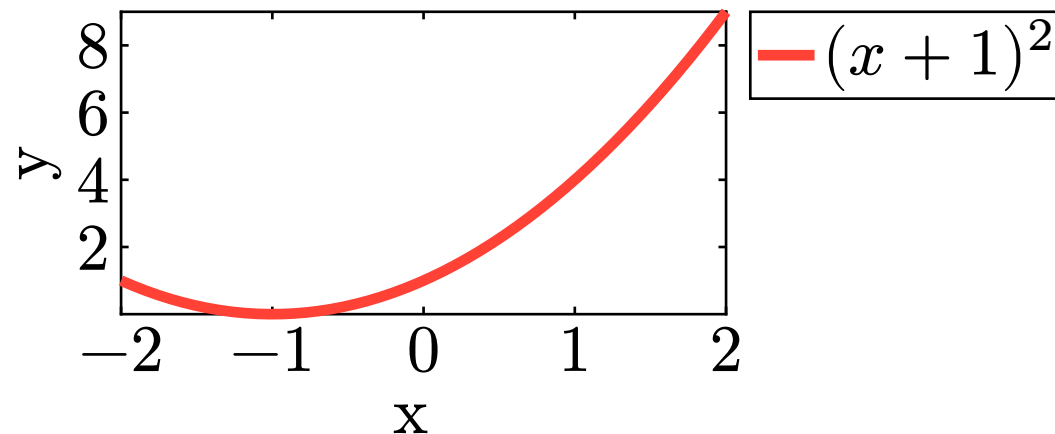
$$\mathcal{L}(\mathbf{x}, \mathbf{y}, \theta) = \sum_{i=1}^n \text{error}(f(x_{[i]}, \theta), y_{[i]}) = \sum_{i=1}^n (f(x_{[i]}, \theta) - y_{[i]})^2$$

Review

We wanted the parameters that minimized the loss function

We introduced the arg min operator

$$f(x) = (x + 1)^2$$



$$\arg \min_x f(x) = -1$$

Review

With the arg min operator, we formally wrote our optimization objective

$$\begin{aligned}\arg \min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) &= \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^n \text{error}\left(f\left(x_{[i]}, \boldsymbol{\theta}\right), y_{[i]}\right) \\ &= \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^n \left(f\left(x_{[i]}, \boldsymbol{\theta}\right) - y_{[i]}\right)^2\end{aligned}$$

Review

We defined the design matrix $\overline{\mathbf{X}}$

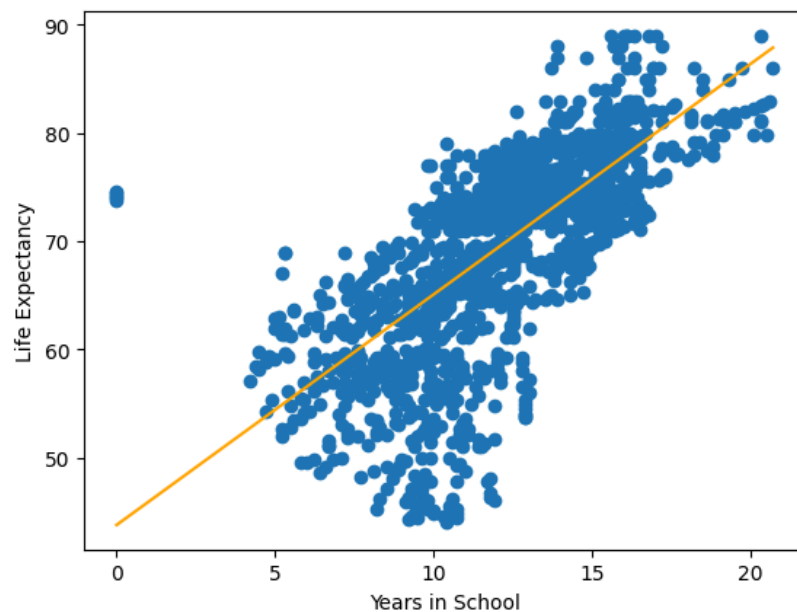
$$\overline{\mathbf{X}} = [\mathbf{1} \ \mathbf{x}] = \begin{bmatrix} 1 & x_{[1]} \\ 1 & x_{[2]} \\ \vdots & \vdots \\ 1 & x_{[n]} \end{bmatrix}$$

We use the design matrix to find an **analytical** solution to the optimization objective

$$\boldsymbol{\theta} = \left(\overline{\mathbf{X}}^\top \overline{\mathbf{X}} \right)^{-1} \overline{\mathbf{X}}^\top \mathbf{y}$$

Review

With this analytical solution, we were able to learn a linear model



Review

Then, we used a trick to extend linear regression to nonlinear models

$$\overline{\mathbf{X}} = \begin{bmatrix} 1 & x_{[1]} \\ 1 & x_{[2]} \\ \vdots & \vdots \\ 1 & x_{[n]} \end{bmatrix} \Rightarrow \overline{\mathbf{X}} = \begin{bmatrix} 1 & \log(1 + x_{[1]}) \\ 1 & \log(1 + x_{[2]}) \\ \vdots & \vdots \\ 1 & \log(1 + x_{[n]}) \end{bmatrix}$$

Review

We extended to polynomials, which are **universal function approximators**

$$\overline{\mathbf{X}} = \begin{bmatrix} 1 & x_{[1]} \\ 1 & x_{[2]} \\ \vdots & \vdots \\ 1 & x_{[n]} \end{bmatrix} \Rightarrow \overline{\mathbf{X}} = \begin{bmatrix} 1 & x_{[1]} & \dots & x_{[1]}^m \\ 1 & x_{[2]} & \dots & x_{[2]}^m \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{[3]} & \dots & x_{[3]}^m \end{bmatrix}$$

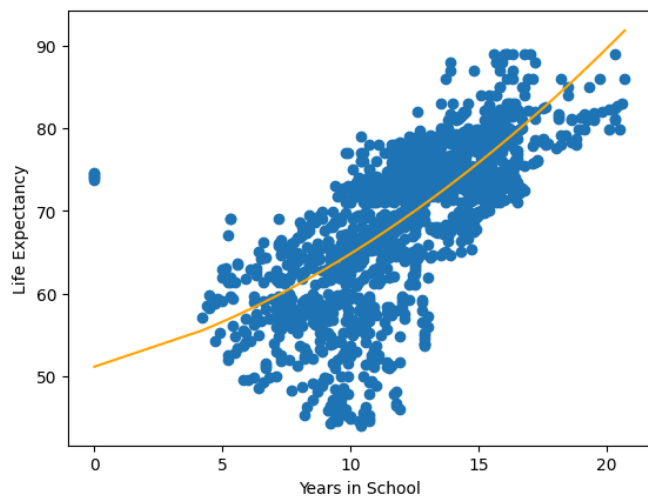
$$f : X \times \Theta \mapsto \mathbb{R}$$

$$\Theta = \mathbb{R}^2 \Rightarrow \Theta = \mathbb{R}^{m+1}$$

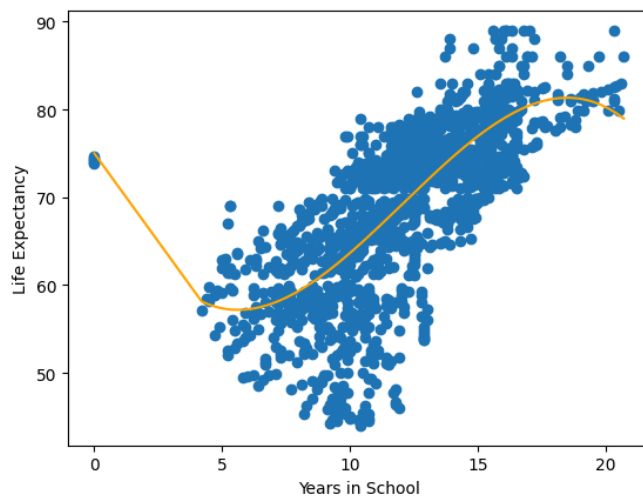
Review

Finally, we discussed overfitting

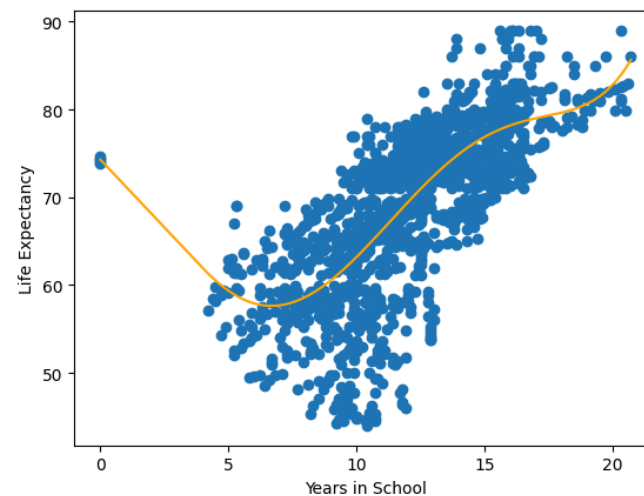
$$f(x, \theta) = \theta_0 + \theta_1 x + \dots + \theta_m x^m$$



$m = 2$



$m = 3$



$m = 5$

Review

We care about **generalization** in machine learning

Always split our dataset into a training dataset and a testing dataset



Multivariate Linear Regression

Multivariate Linear Regression

Last time, we assumed a single-input system

Years of education: $X = \mathbb{R}_+$

But sometimes we want to consider multiple input dimensions

Years of education, BMI, GDP: $X = \mathbb{R}_+^3$

We can solve these problems using linear regression too

We call it **multivariate linear regression**

Multivariate Linear Regression

For multivariate problems, we will define the input dimension as d_x

$$\mathbf{x} \in X; \quad X = \mathbb{R}^{d_x}$$

We write a single input as

$$\mathbf{x}_{[i]} = \begin{bmatrix} x_{[i],1} \\ x_{[i],2} \\ \vdots \\ x_{[i],d_x} \end{bmatrix}$$

$x_{[i],1}$ refers to the first dimension of datapoint i

Multivariate Linear Regression

The design matrix for a multivariate linear system is

$$\overline{\mathbf{X}} = \begin{bmatrix} 1 & x_{[1],1} & x_{[1],2} & \cdots & x_{[1],d_x} \\ 1 & x_{[2],1} & x_{[2],2} & \cdots & x_{[2],d_x} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{[n],1} & x_{[n],2} & \cdots & x_{[n],d_x} \end{bmatrix}$$

Remember $x_{[n],d_x}$ refers to dimension d_x of training data n

The solution is the same as before

$$\theta = \left(\overline{\mathbf{X}}^\top \overline{\mathbf{X}} \right)^{-1} \overline{\mathbf{X}}^\top \mathbf{y}$$

Limitations of Linear Regression

Limitations of Linear Regression

Linear models are useful for certain problems

1. Analytical solution
2. Low data requirement

Issues arise with other problems

1. Poor scalability
2. Polynomials do not generalize well

Limitations of Linear Regression

Issues arise with other problems

1. **Poor scalability**
2. Polynomials do not generalize well

Limitations of Linear Regression

So far, we have seen:

One-dimensional polynomial functions

$$\overline{\mathbf{X}} = \begin{bmatrix} 1 & x_{[1]} & \dots & x_{[1]}^m \\ 1 & x_{[2]} & \dots & x_{[2]}^m \\ \vdots & \vdots & \ddots & \\ 1 & x_{[3]} & \dots & x_{[3]}^m \end{bmatrix}$$

Multi-dimensional linear functions

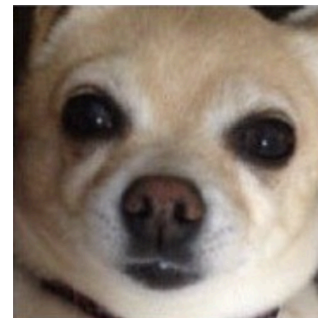
$$\overline{\mathbf{X}} = \begin{bmatrix} 1 & x_{[1],1} & x_{[1],2} & \dots & x_{[1],d_x} \\ 1 & x_{[2],1} & x_{[2],2} & \dots & x_{[2],d_x} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{[n],1} & x_{[n],2} & \dots & x_{[n],d_x} \end{bmatrix}$$

Combine them to create multi-dimensional polynomial functions

Limitations of Linear Regression

Let us do an example

Task: predict how many ❤️ a photo gets on social media



$f : X \times \Theta \mapsto Y; \quad X : \text{Image}, \quad Y : \text{Number of } \heartsuit$

$$X = \mathbb{Z}_+^{256 \times 256} = \mathbb{Z}_+^{65536}$$

$$Y = \mathbb{Z}_+$$

Highly nonlinear task, use a polynomial with order $m = 20$

Limitations of Linear Regression

$$\overline{\mathbf{X}} = \begin{bmatrix} \mathbf{x}_{[1],d_x} & \cdots & \mathbf{x}_{[n],d_x} \end{bmatrix}^\top$$

$$\mathbf{x}_{[i]} = \begin{bmatrix} 1 & x_{[i],1} & x_{[i],2} & \cdots & x_{[i],d_x} & \cdots & \underbrace{x_{[i],1}x_{[i],1}}_{x_{[i],1}^2} & x_{[i],2}x_{[i],1} & \cdots & x_{[i],d_x} & \cdots & x_{[i],d_x}^m \end{bmatrix}$$

Question: How many columns in this matrix?

Hint: Consider a multivariate polynomial (e.g., 2 variables, order 3)

$$d_x = 2, m = 3: x^3 + y^3 + x^2y + y^2x + x^2 + y^2 + xy + x + y + 1$$

Hint: $d_x = 2, m = 3$ has 9 terms

Answer: $(m)^{d_x} = 20^{655360} \approx 10^{852000}$

Limitations of Linear Regression

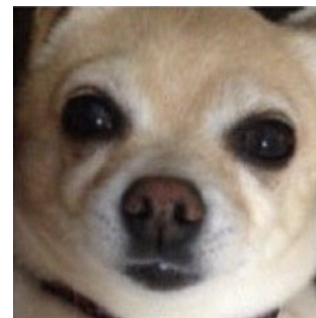
How big is 10^{852000} ?

Question: How many atoms are there in the universe?

Answer: 10^{82}

There is not enough matter in the universe to represent one row of $\overline{X}$

We cannot predict how many ❤️
the picture will get



Limitations of Linear Regression

Polynomial regression does not scale to large inputs

1. **Poor scalability**
2. Polynomials do not generalize well

Limitations of Linear Regression

Issues arise with other problems

1. Poor scalability
2. **Polynomials do not generalize well**

Limitations of Linear Regression

What happens to polynomials outside of the support (dataset)?

Take the limit of polynomials to see their behavior

$$\lim_{x \rightarrow \infty} \theta_0 + \theta_1 x + \dots + \theta_m x^m$$

Equation of a polynomial

$$\lim_{x \rightarrow \infty} x^m \left(\frac{\theta_0}{x^m} + \frac{\theta_1}{x^{m-1}} + \dots + \theta_m \right)$$

Factor out x^m

$$\lim_{x \rightarrow \infty} x^m \cdot \lim_{x \rightarrow \infty} \left(\frac{\theta_0}{x^m} + \frac{\theta_1}{x^{m-1}} + \dots + \theta_m \right) \text{ Split the limit (limit of products)}$$

Limitations of Linear Regression

$$\lim_{x \rightarrow \infty} x^m \cdot \lim_{x \rightarrow \infty} \left(\frac{\theta_0}{x^m} + \frac{\theta_1}{x^{m-1}} + \dots + \theta_m \right) \text{ Split the limit (limit of products)}$$

$$\left(\lim_{x \rightarrow \infty} x^m \right) \cdot (0 + 0 + \dots + \theta_m) \quad \text{Evaluate right limit}$$

$$\theta_m \lim_{x \rightarrow \infty} x^m \quad \text{Rewrite}$$

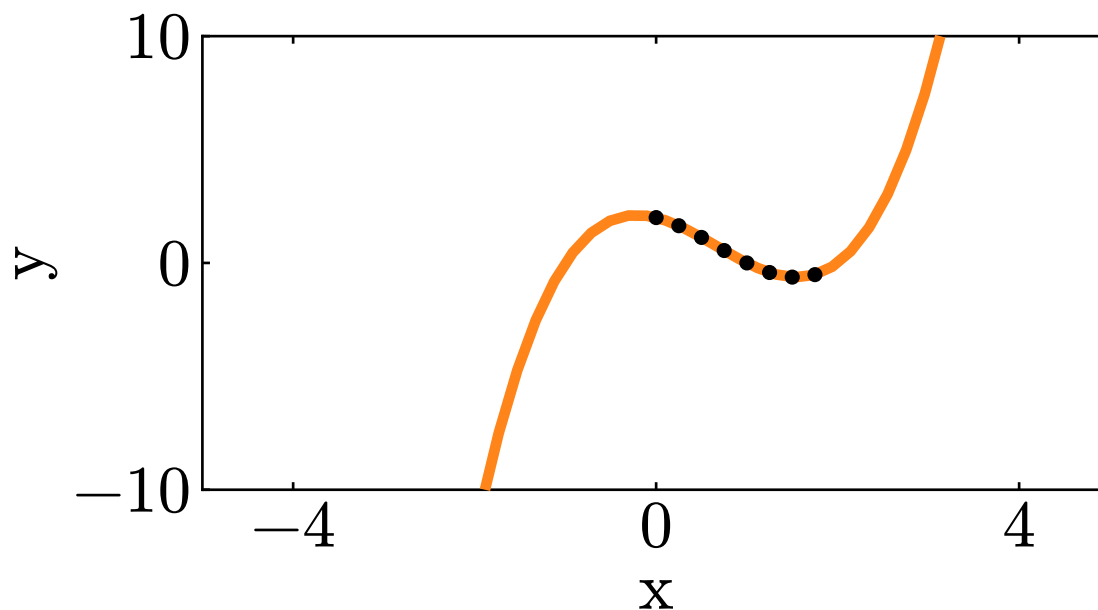
$$\theta_m \lim_{x \rightarrow \infty} x^m = \infty \quad \text{If } \theta_m > 0$$

$$\theta_m \lim_{x \rightarrow \infty} x^m = -\infty \quad \text{If } \theta_m < 0$$

Limitations of Linear Regression

Polynomials quickly tend towards $-\infty, \infty$ outside of the support

$$f(x) = 2 - x - 2x^2 + x^3$$



Remember, to predict new data f must generalize

Limitations of Linear Regression

Linear regression has issues

1. Poor scalability
2. Polynomials do not generalize well

Limitations of Linear Regression

We can use neural networks as an alternative to linear regression

Neural network benefits:

1. Scale to large inputs
2. Slightly better generalization

Drawbacks:

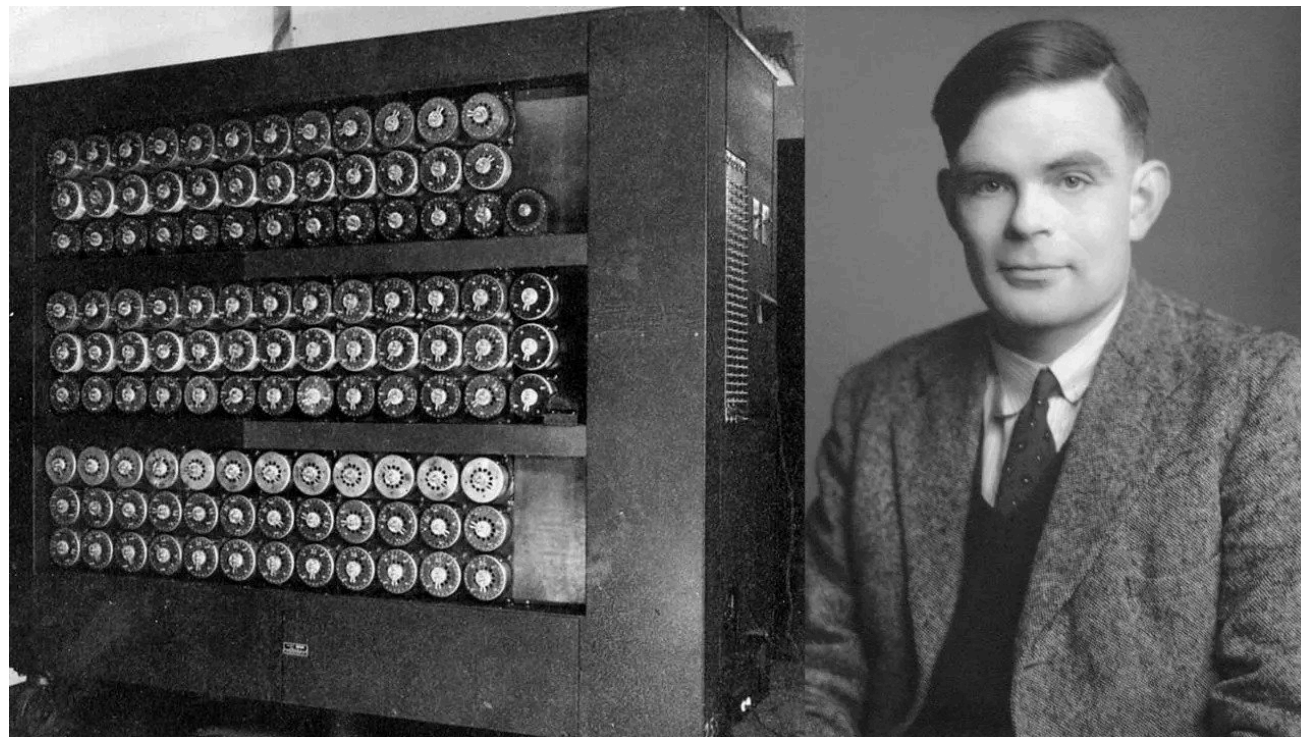
1. No analytical solution
2. Big data requirement

History of Neural Networks

History of Neural Networks

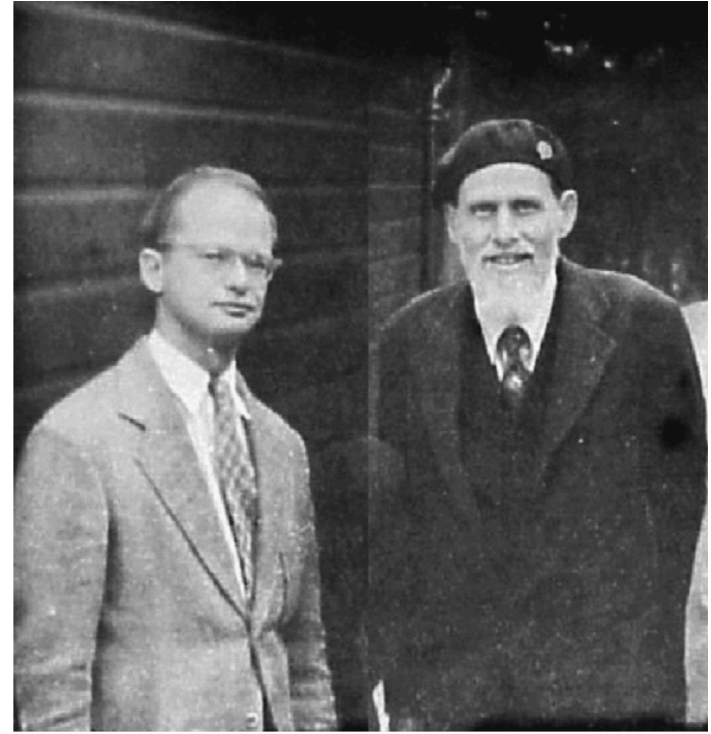
In 1939-1945, there was a World War (抗日战争)

Militaries invested funding for research, and invented the computer



History of Neural Networks

Meanwhile, a neuroscientist and mathematician (McCullough and Pitts) were trying to understand the human brain

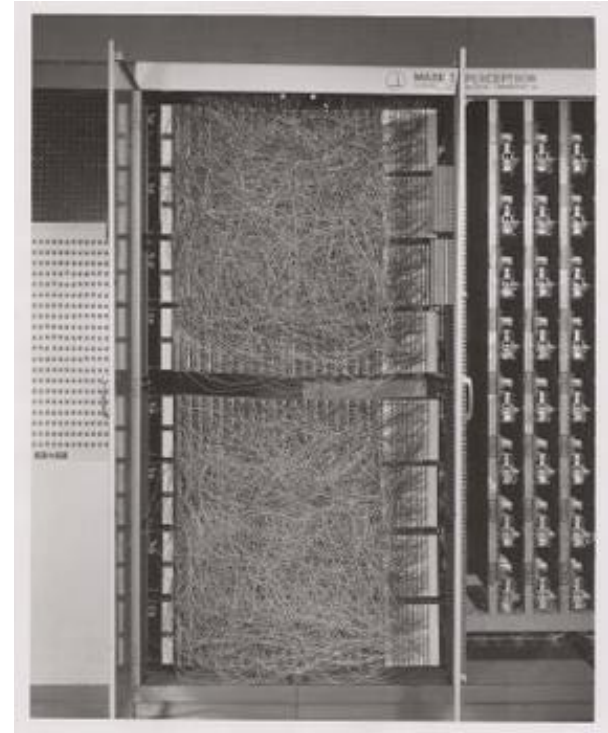


They designed the theory for the first neural network

History of Neural Networks

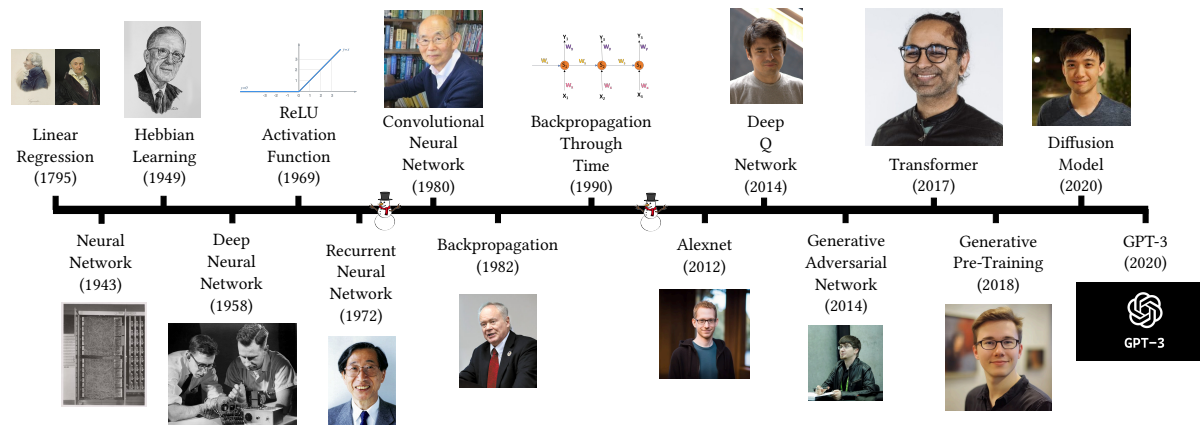
Rosenblatt implemented this neural network theory on a computer a few years later

At the time, computers were very slow and expensive



History of Neural Networks

Through advances in theory and hardware, neural networks became slightly better



Around 2012, these improvements culminated in neural networks that perform like humans

History of Neural Networks

So what is a neural network?

It is a function, inspired by how the brain works

$$f : X \times \Theta \mapsto Y$$

History of Neural Networks

Brains and neural networks rely on **neurons**

Brain: Biological neurons → Biological neural network

Computer: Artificial neurons → Artificial neural network

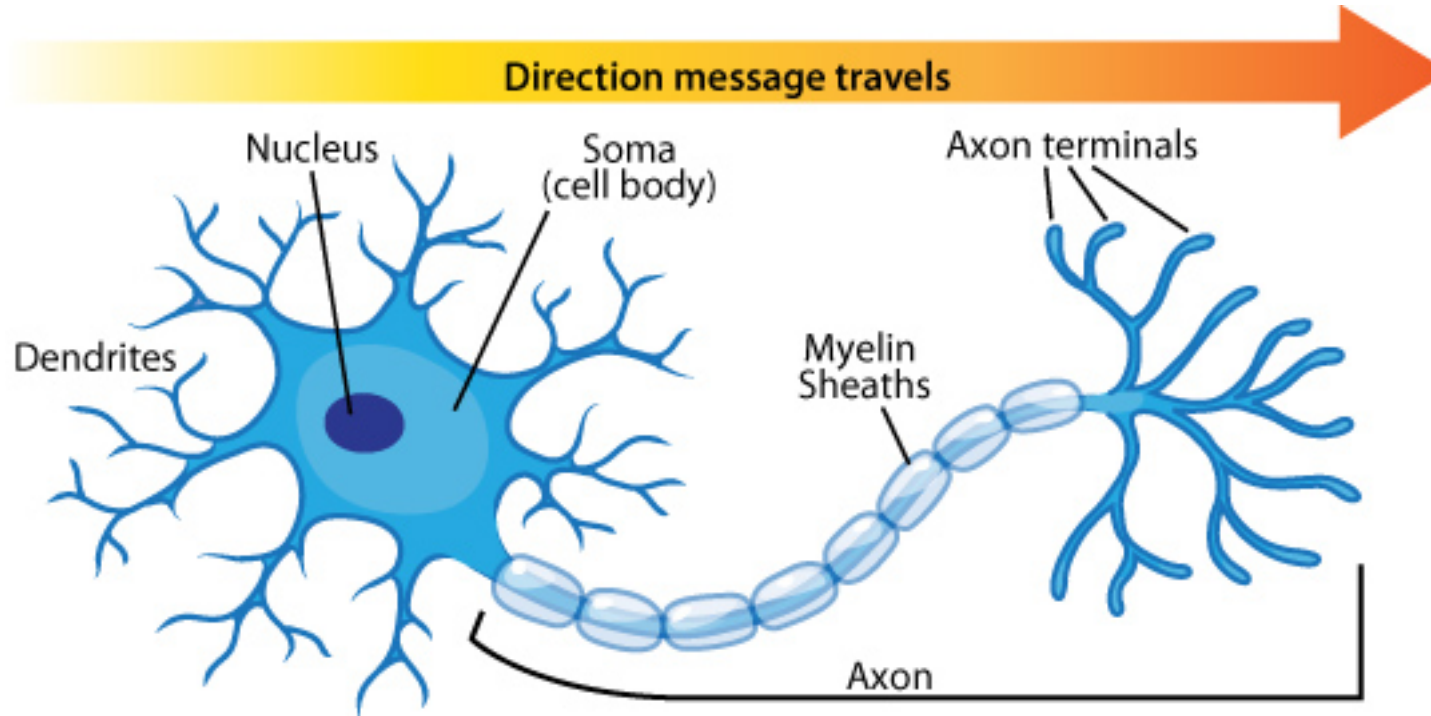
To understand artificial neurons, we must first understand human neurons

Note: I am not a neuroscientist

- I may make simplifications or errors with biology
- Neuroscience students please correct me!

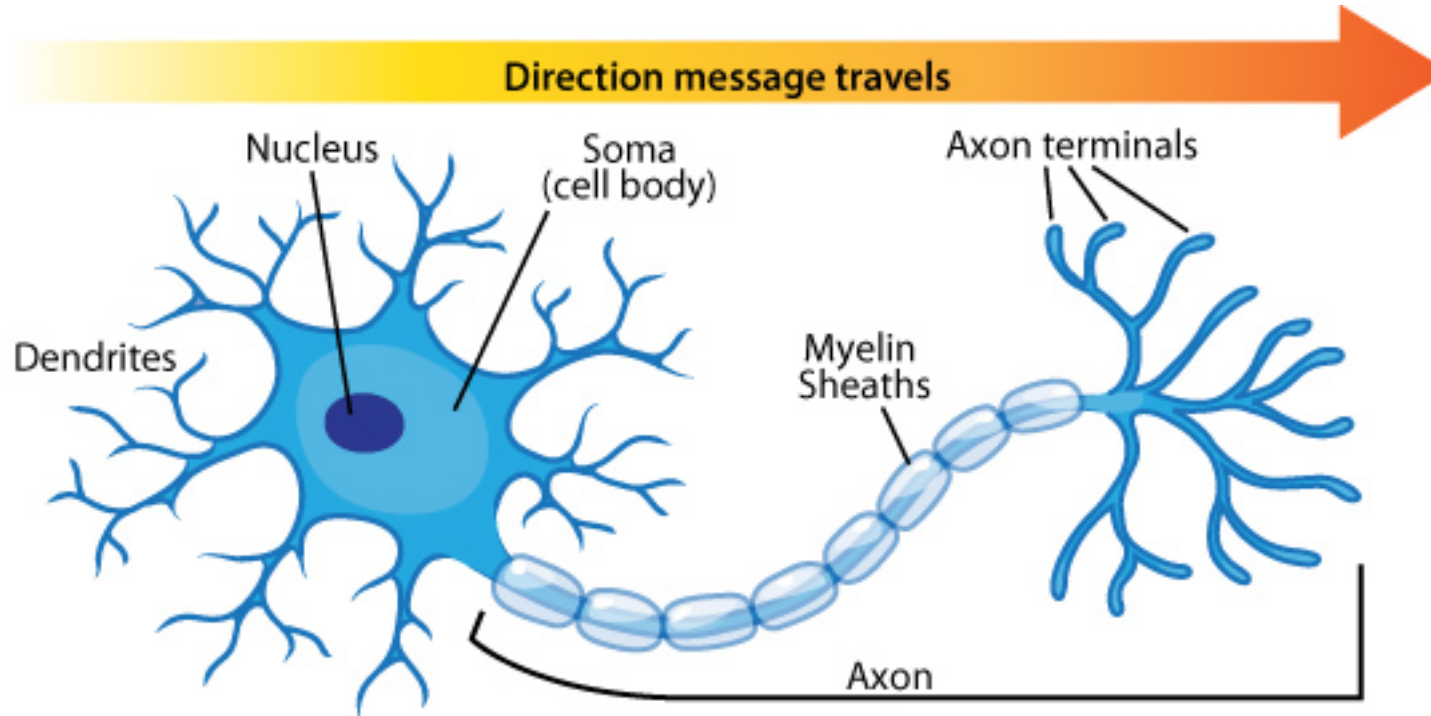
Biological Neurons

Biological Neurons



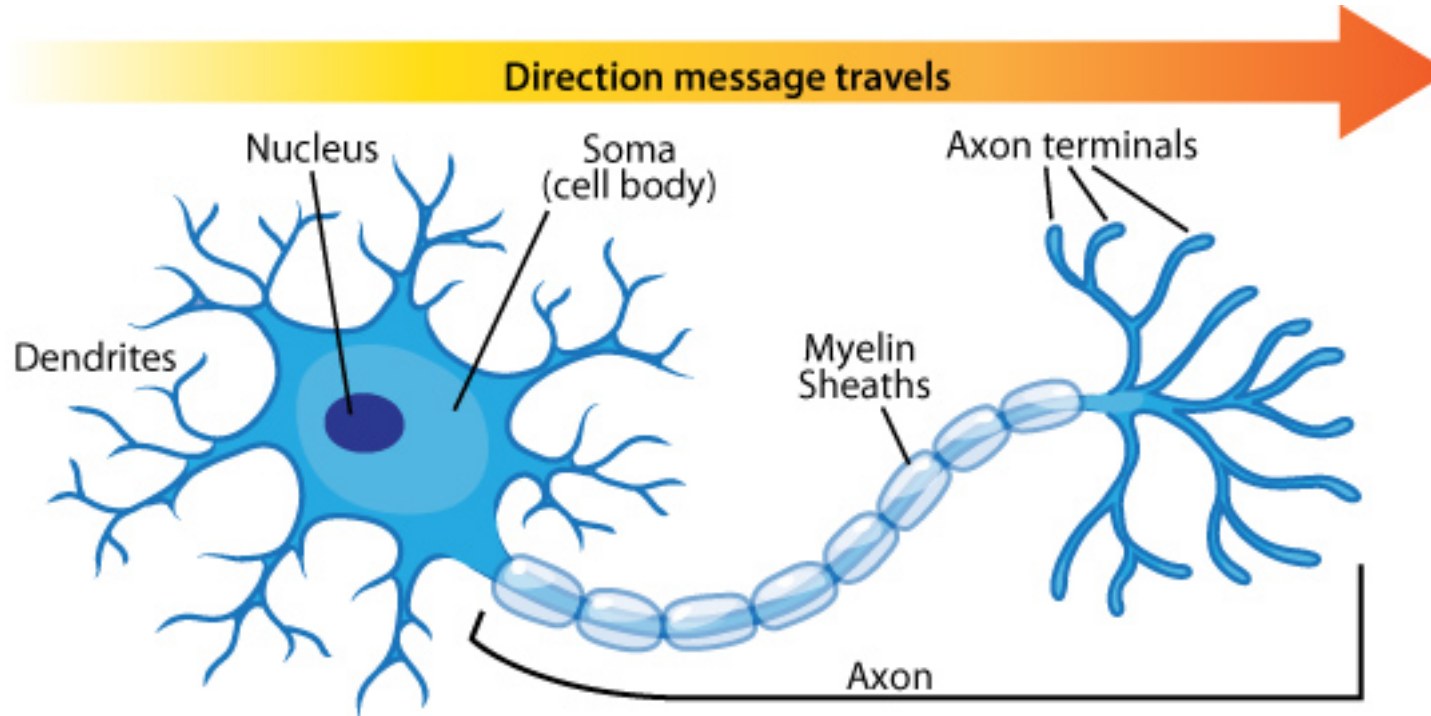
A simplified neuron consists of many parts

Biological Neurons



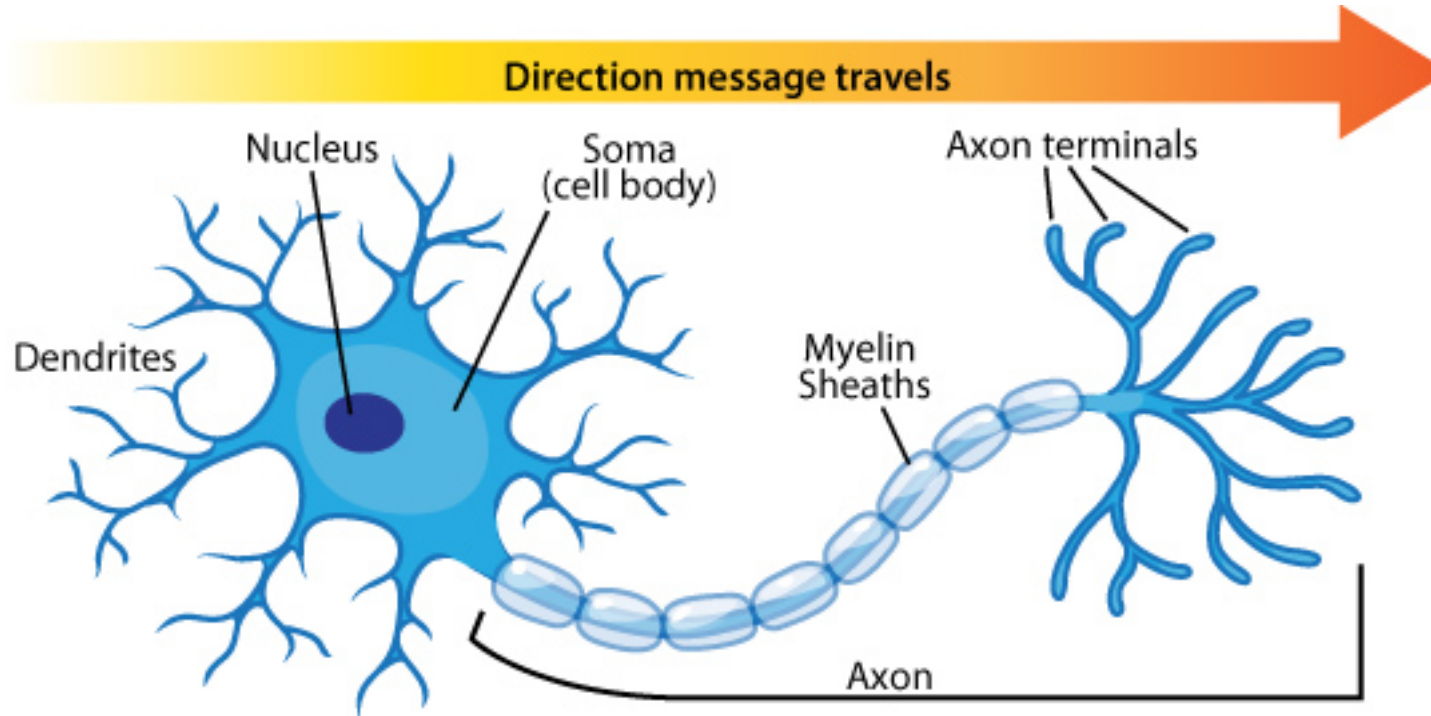
Neurons send messages based on messages received from other neurons

Biological Neurons



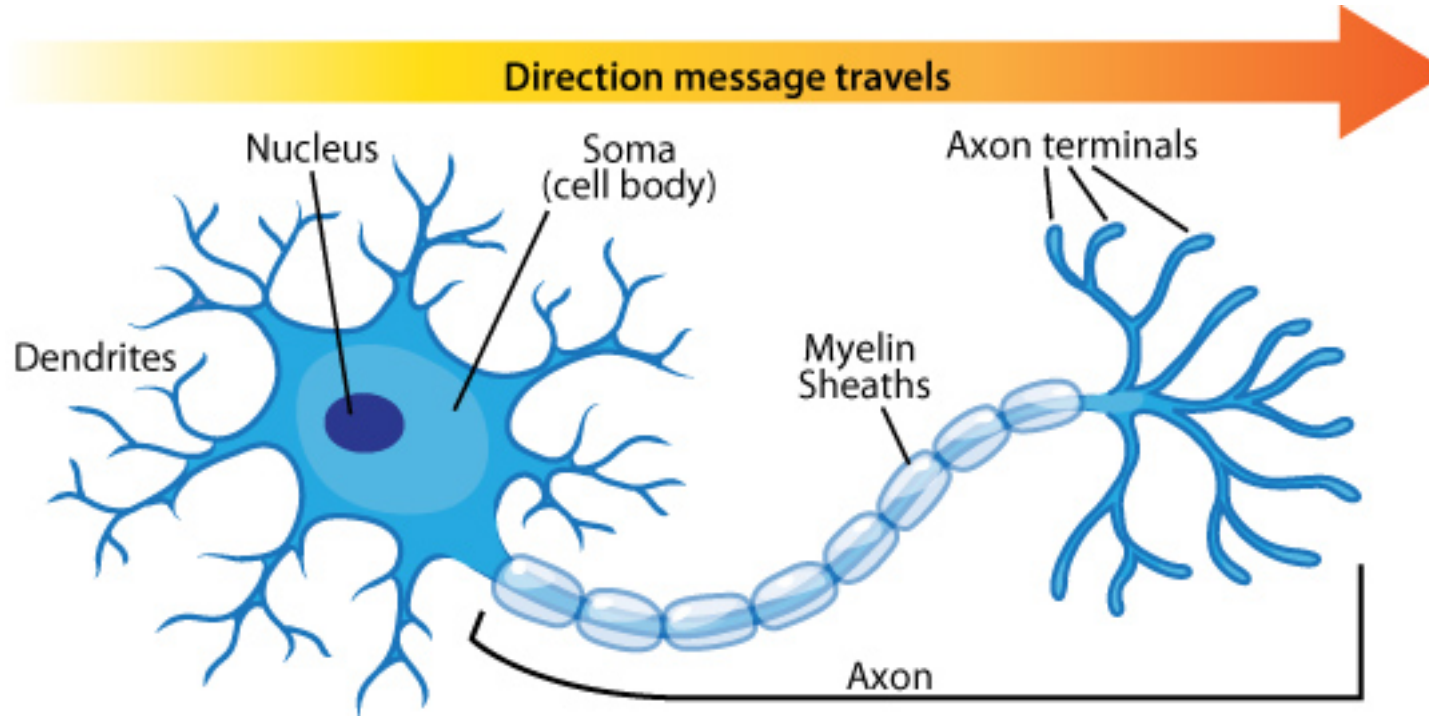
Incoming electrical signals travel along dendrites

Biological Neurons



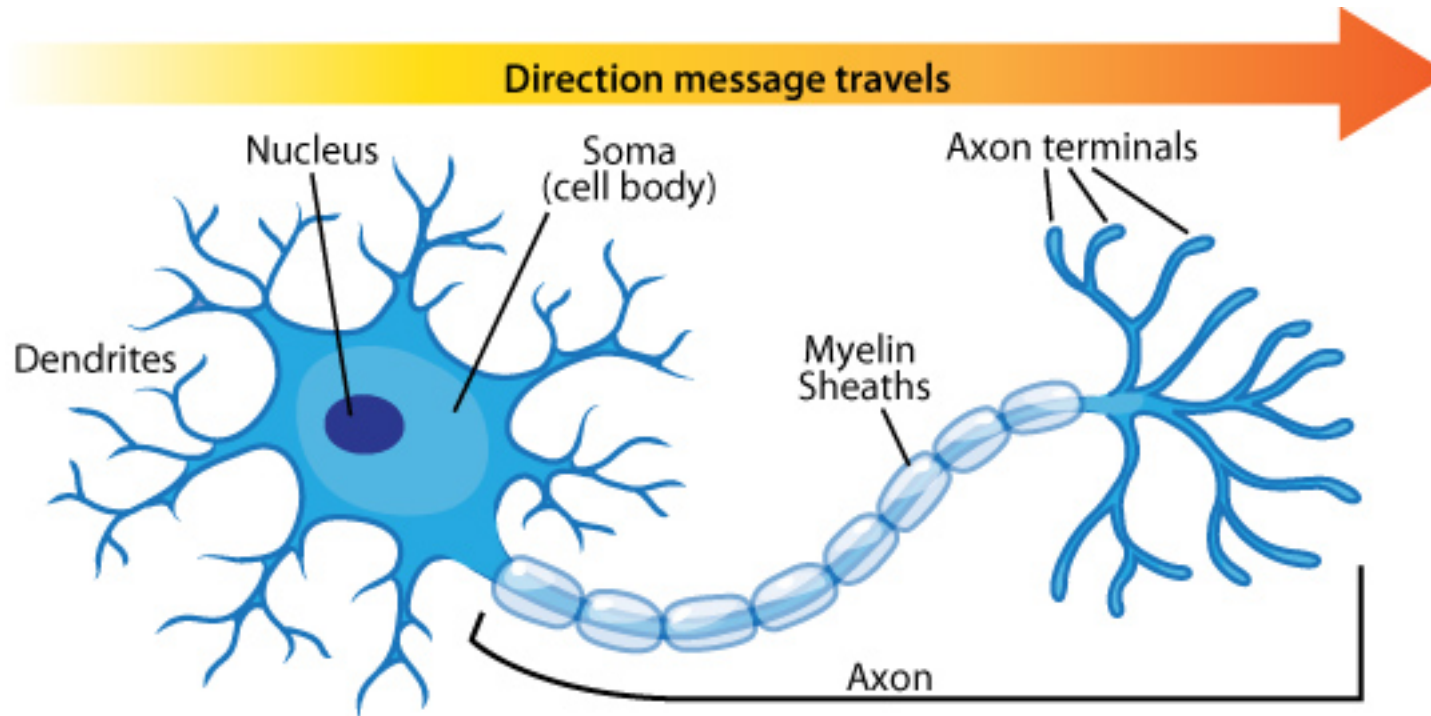
Electrical charges collect in the Soma (cell body)

Biological Neurons



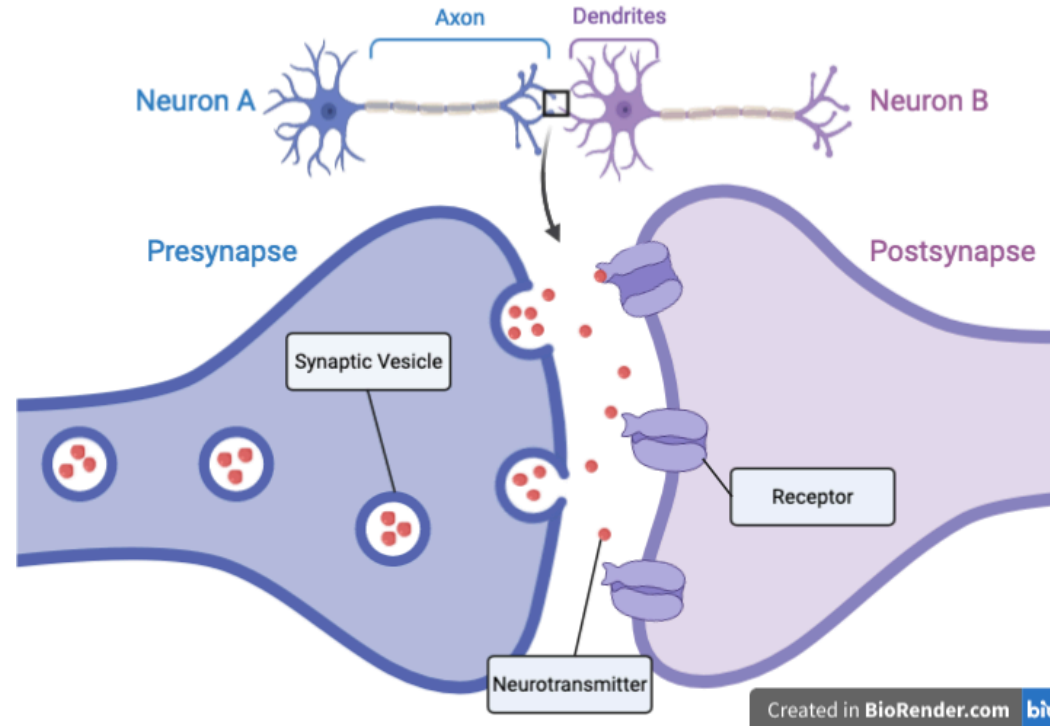
The axon outputs an electrical signal to other neurons

Biological Neurons



The axon terminals will connect to dendrites of other neurons through a synapse

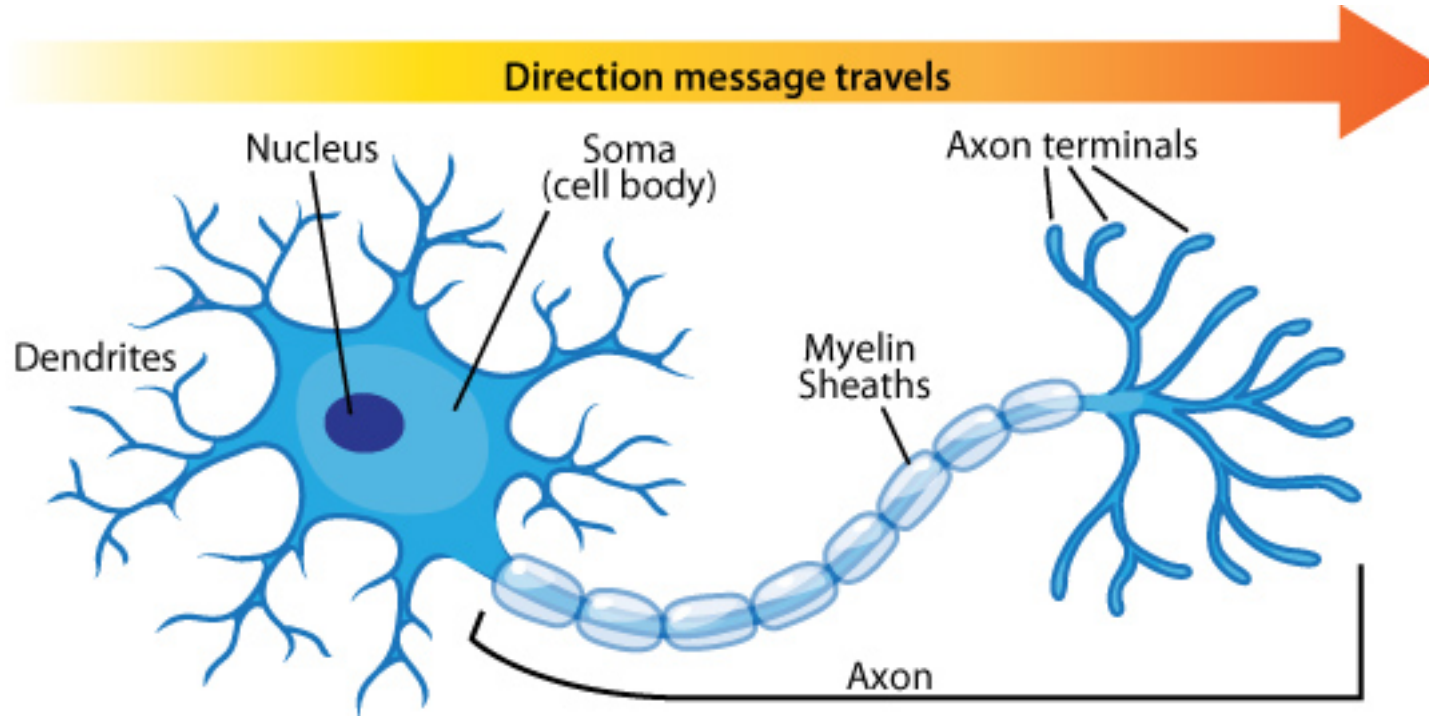
Biological Neurons



Synapse converts electrical signal, to chemical signal, to electrical signal

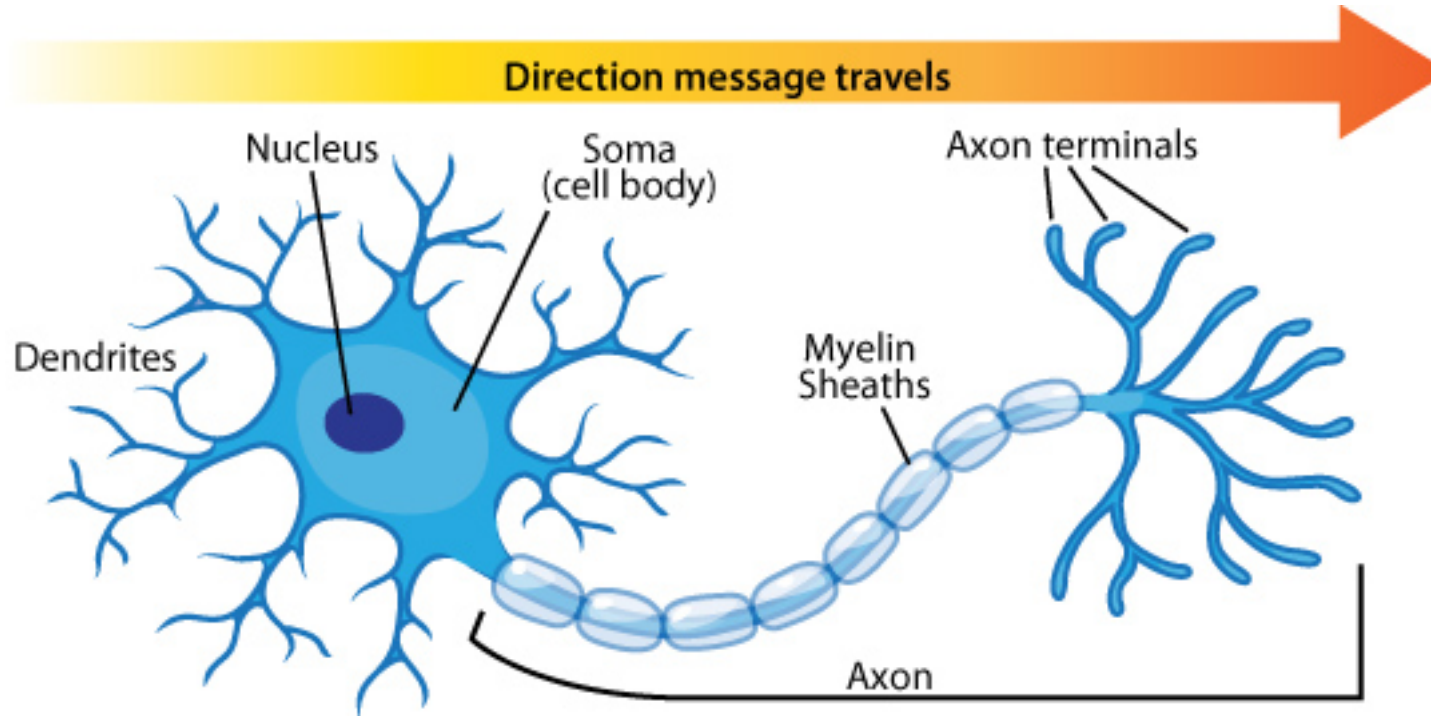
Synaptic weight determines how well a signal crosses the gap

Biological Neurons



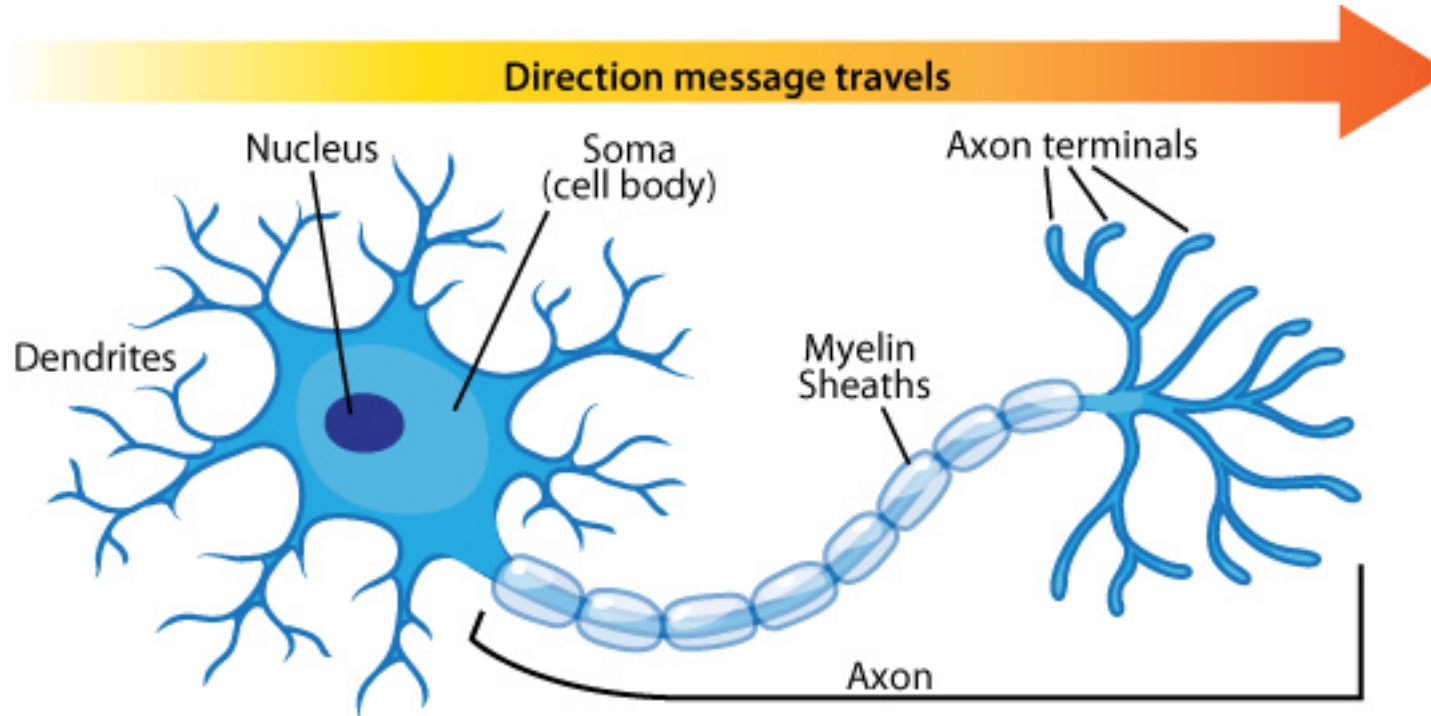
To simplify, we combine axon terminals, synaptic gap, and dendrites

Biological Neurons



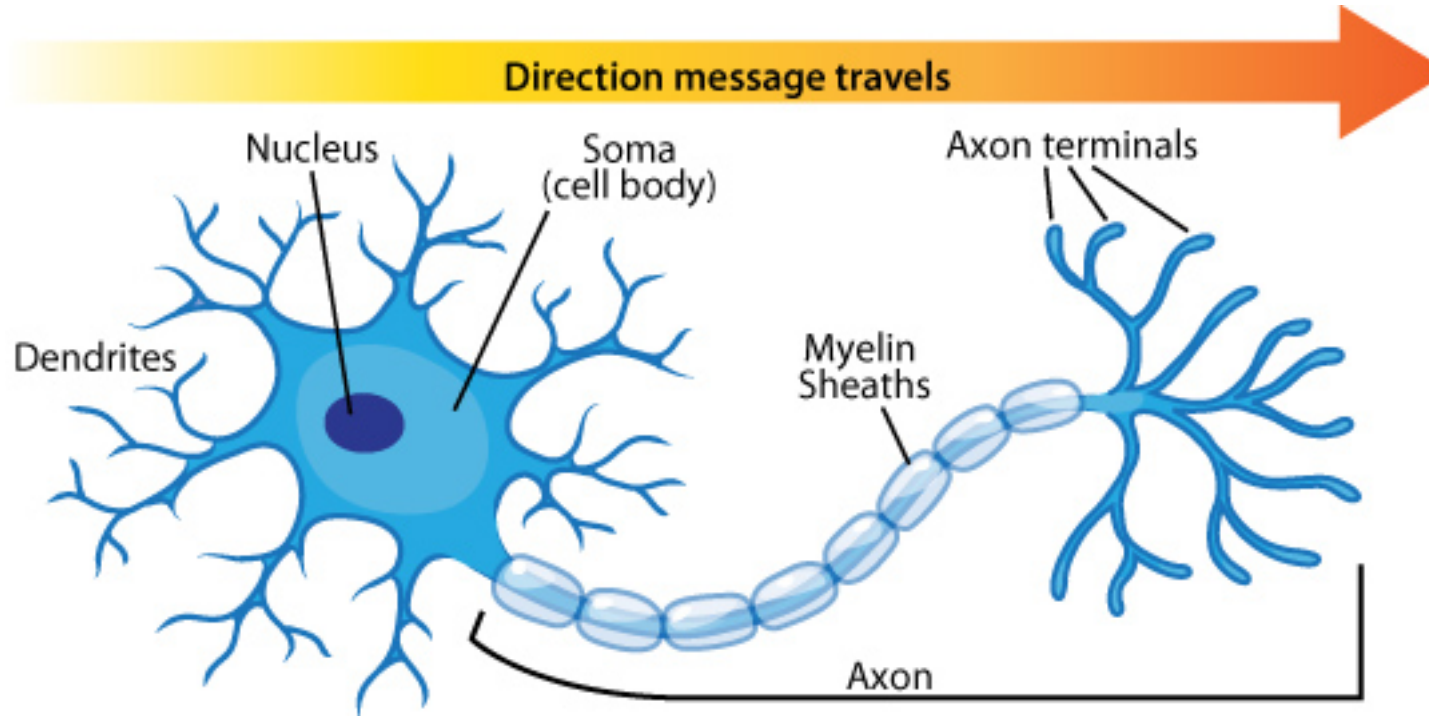
“Simplified” neuron has many dendrites but only one axon terminal

Biological Neurons



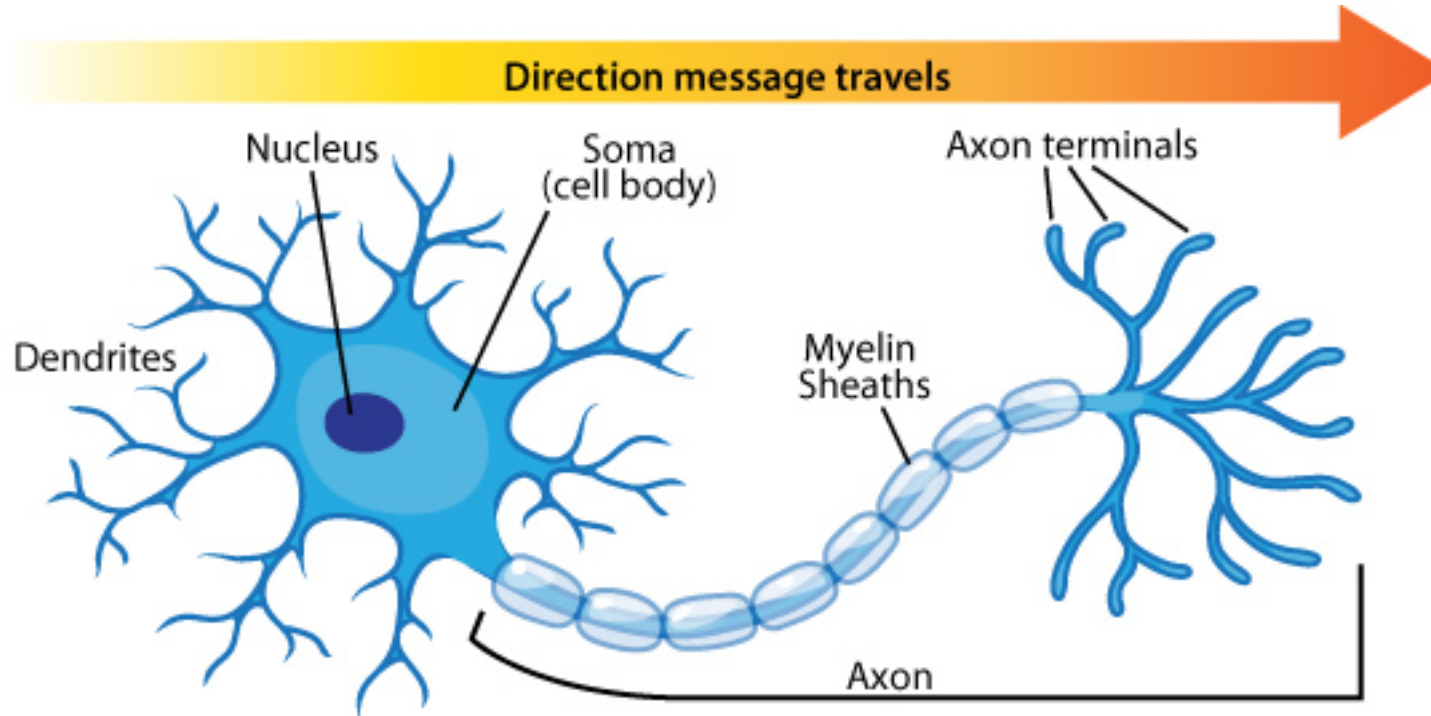
The neuron takes many inputs, and produces a single output

Biological Neurons



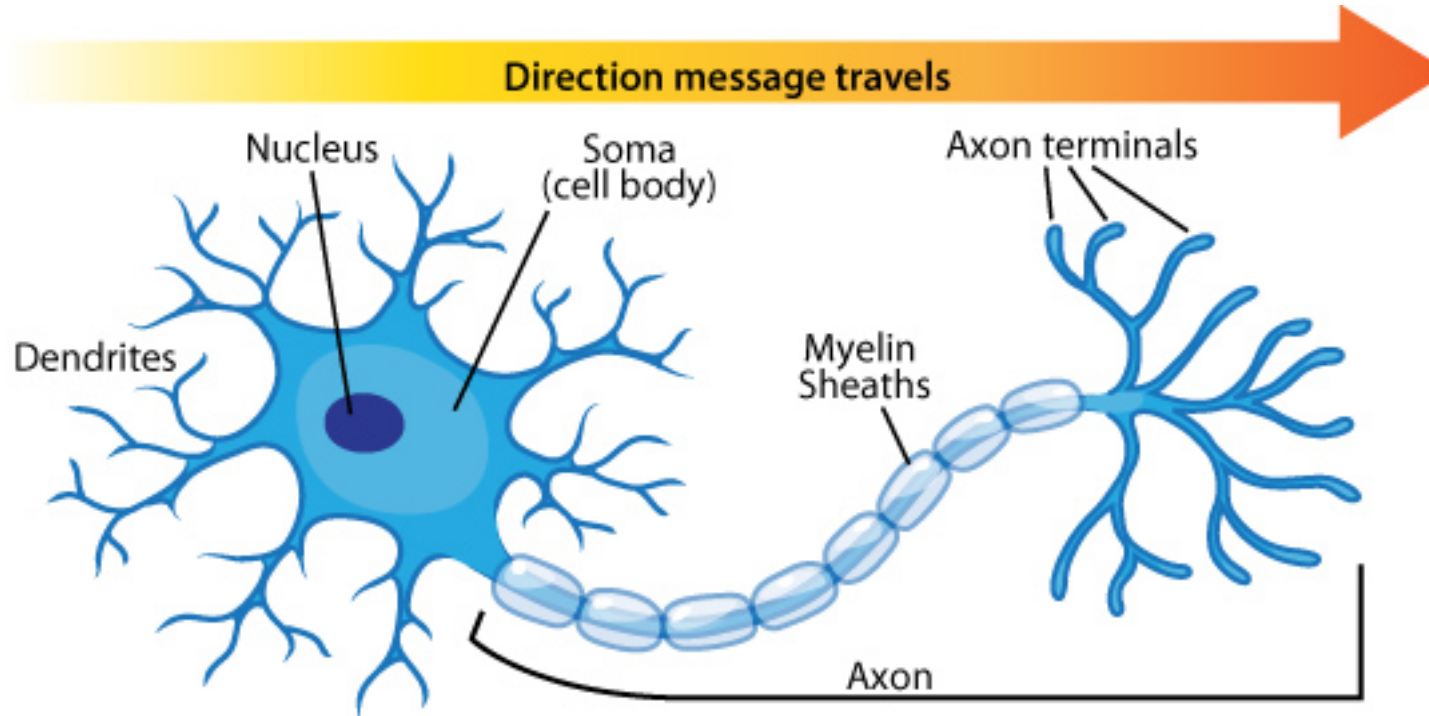
Due to electric potential, signals behave like lightning

Biological Neurons



Potential varies, eventually it will “snap” (fire) like lightning

Biological Neurons

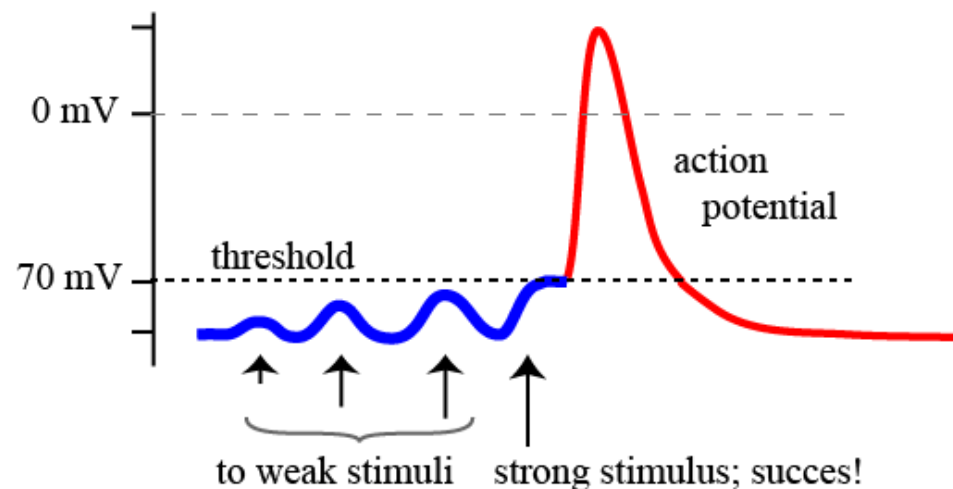


The neuron only fires sometimes, other times it does not fire

Biological Neurons

How does a neuron decide to send an impulse (“fire”)?

Incoming impulses (via dendrites) change the electric potential of the neuron

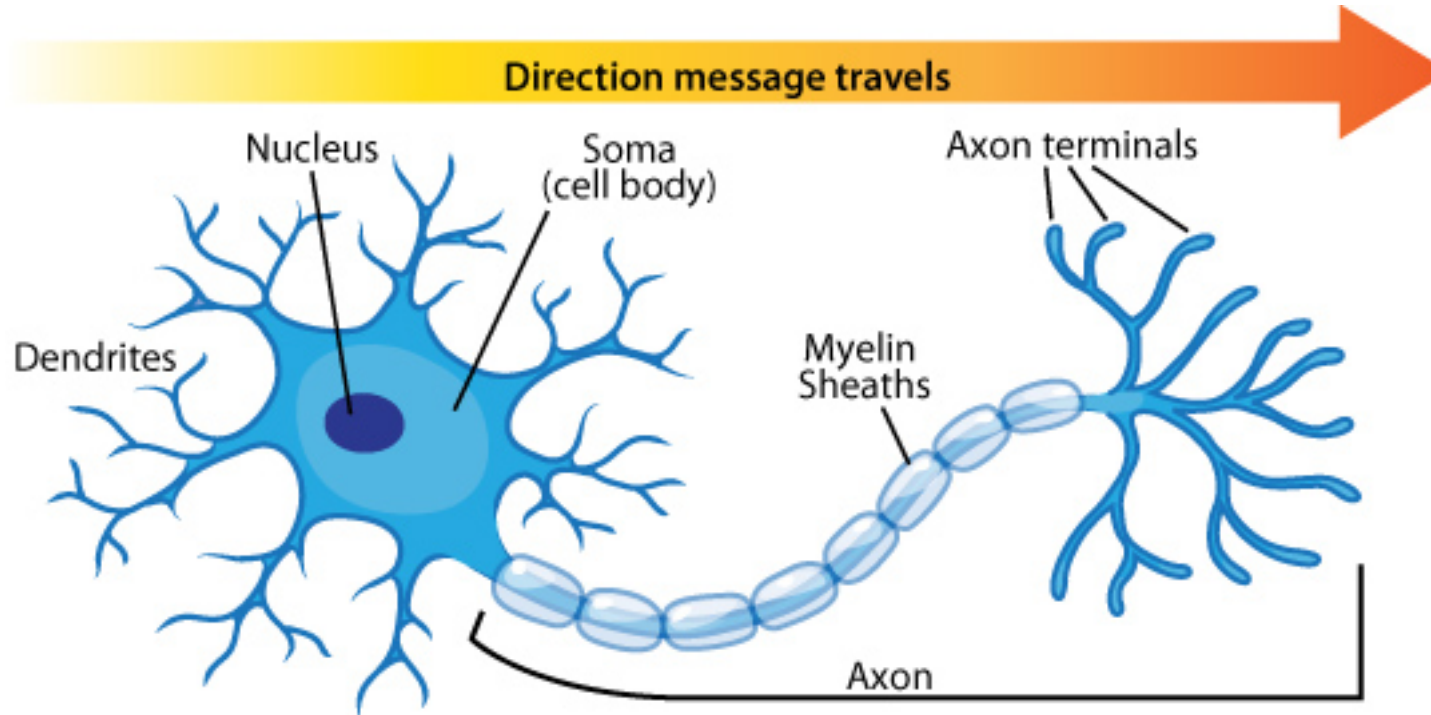


In a parallel circuit, we can sum voltages together

Neural dendrites form a parallel circuit!

Biological Neurons

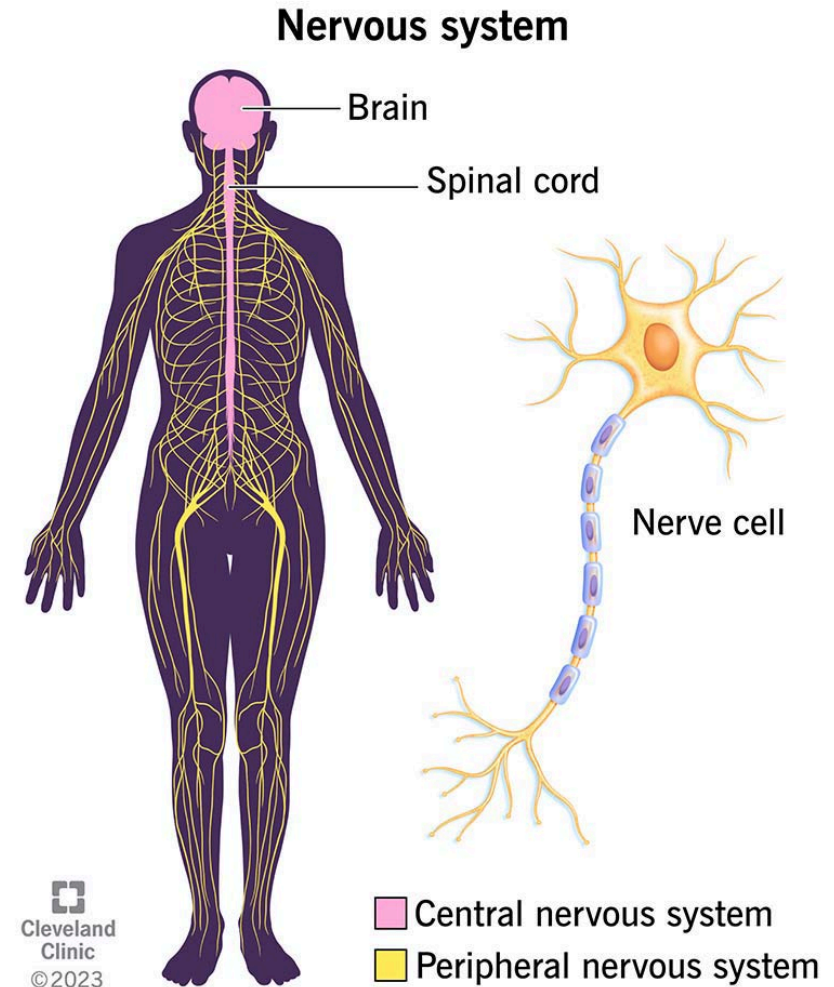
Neural dendrites form a parallel circuit!



Many active dendrites will add together and trigger an impulse

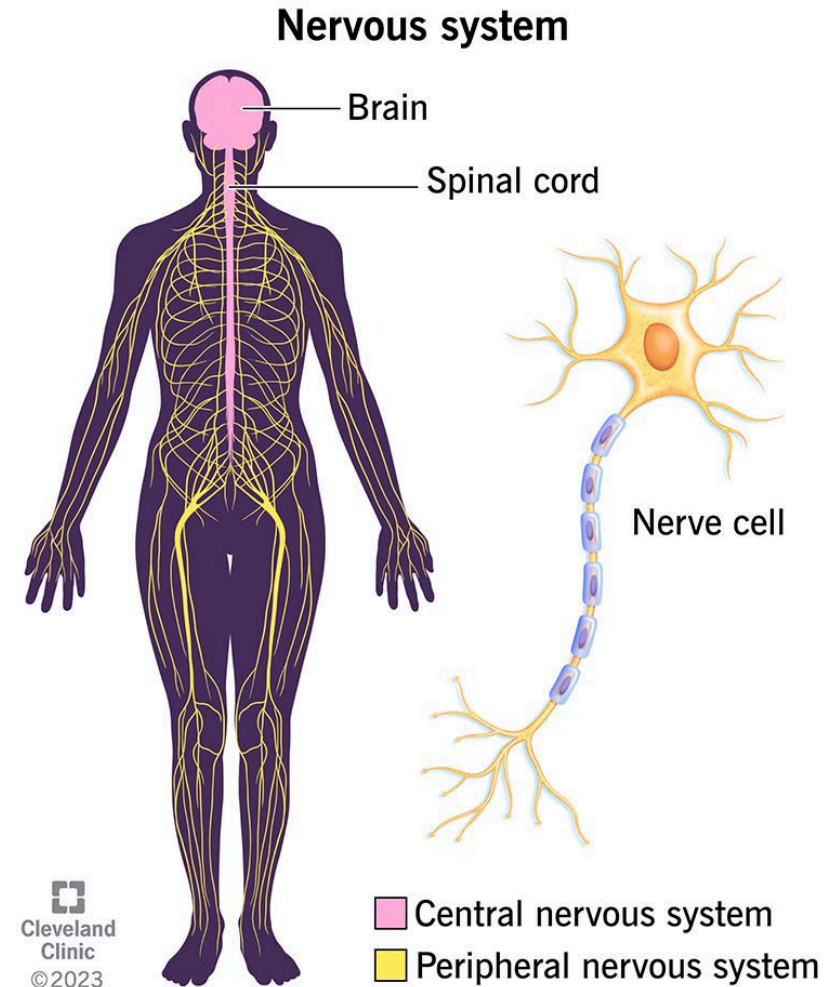
Biological Neurons

Touch triggers initial nerve impulse, starts a chain reaction into the brain



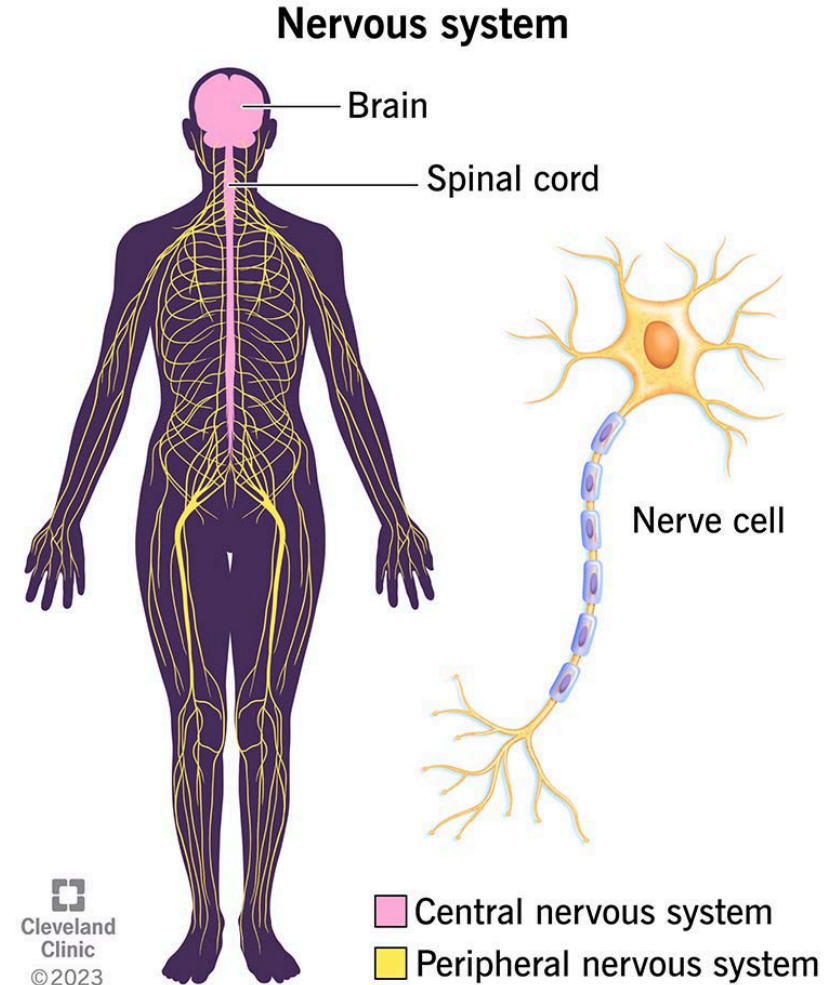
Biological Neurons

When the signal reaches the brain,
we feel the touch



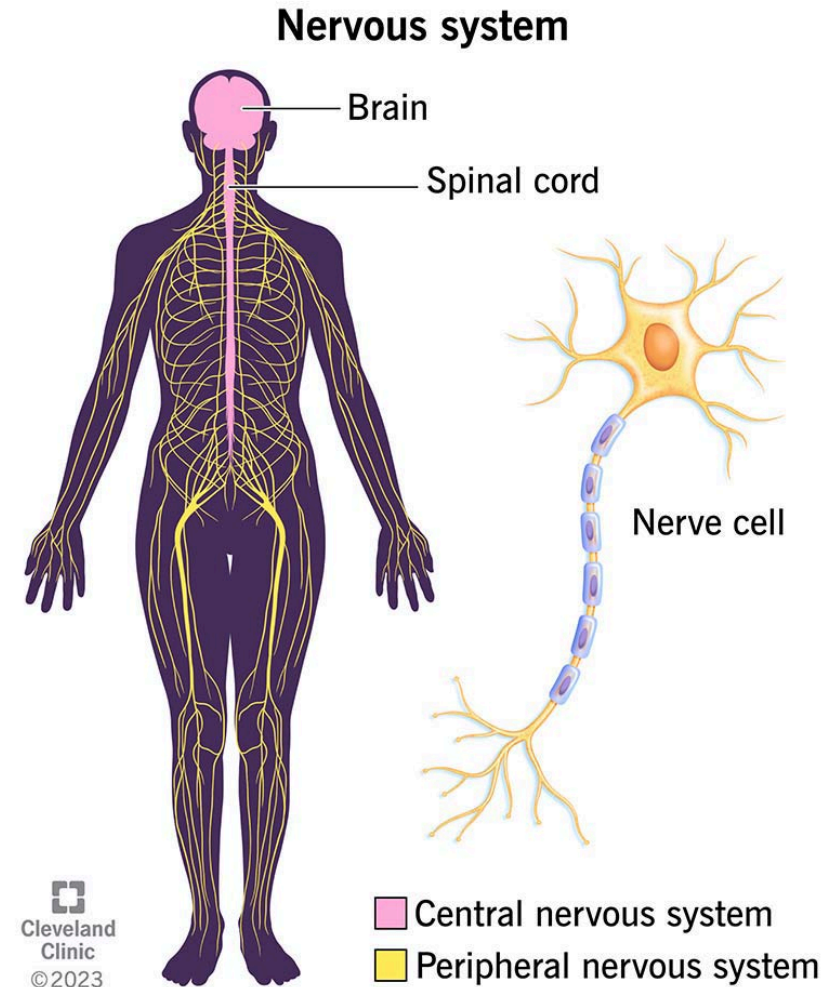
Biological Neurons

Feeling the touch activates other neurons in the brain (thinking)



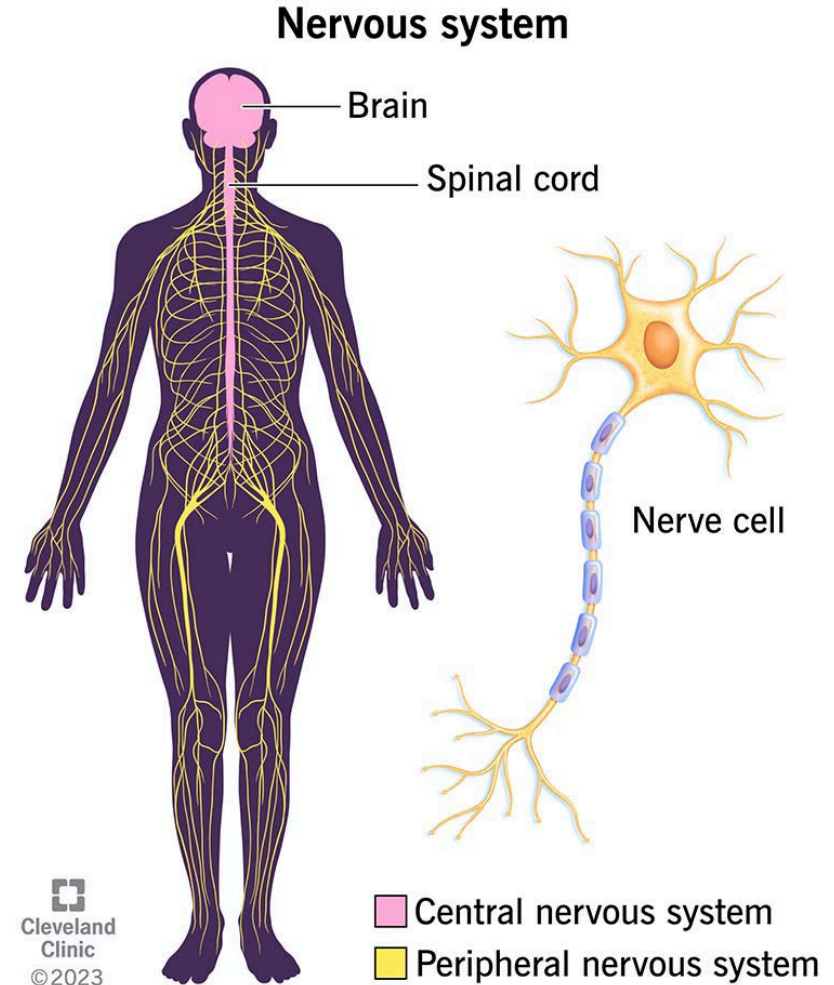
Biological Neurons

After thinking, we will take action



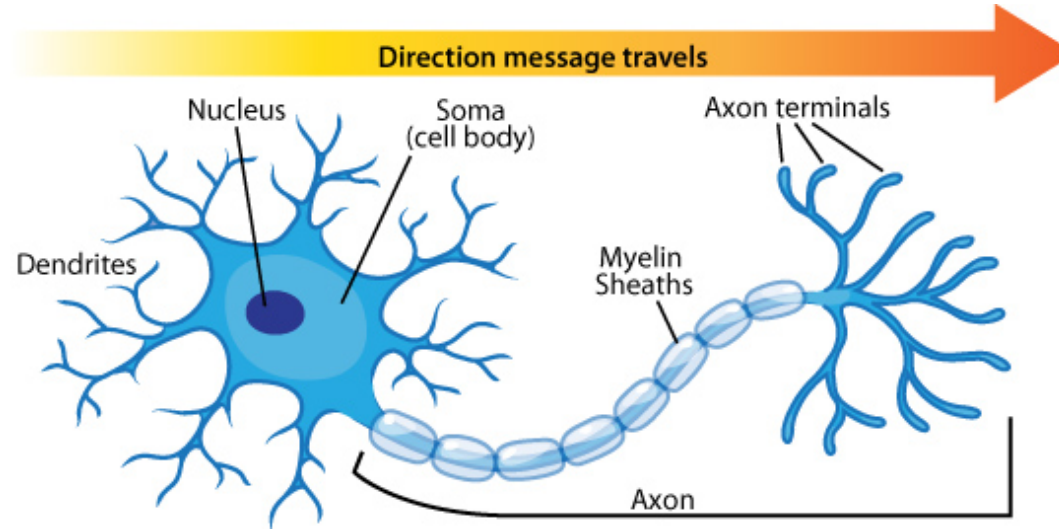
Biological Neurons

Action follows reverse path, leaves the brain and goes to motor neurons



Artificial Neurons

Artificial Neurons



Question: How could we write a neuron as a function? $f : _ \mapsto _$

Answer:

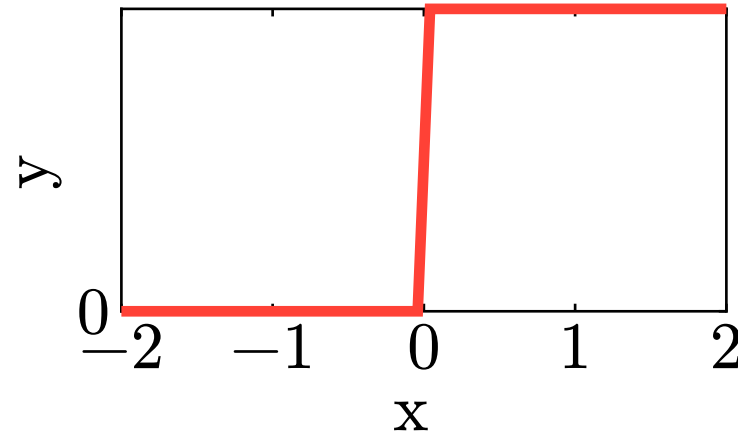
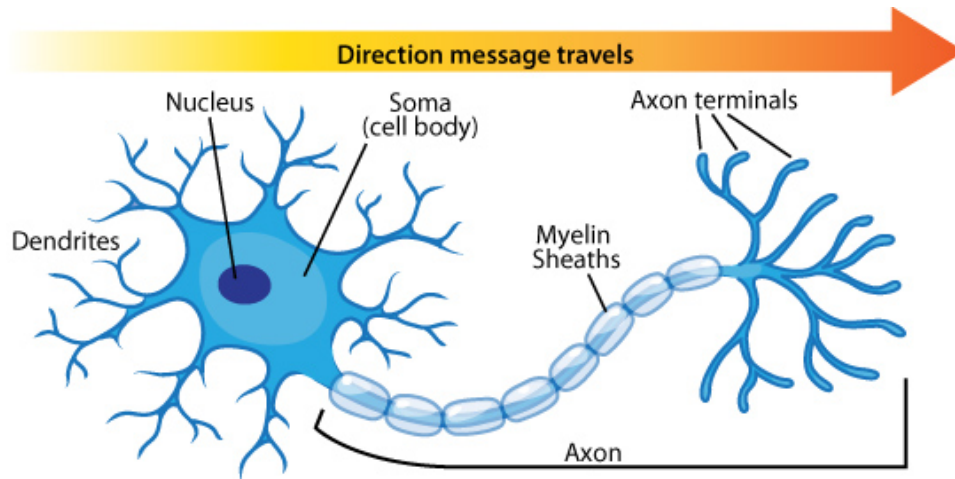
$$f : \underbrace{\mathbb{R}^{d_x}}_{\text{Dendrite voltages}} \times \underbrace{\mathbb{R}^{d_x}}_{\text{Synaptic weight}} \mapsto \underbrace{\mathbb{R}}_{\text{Axon voltage}}$$

Artificial Neurons

Let us implement an artificial neuron as a function

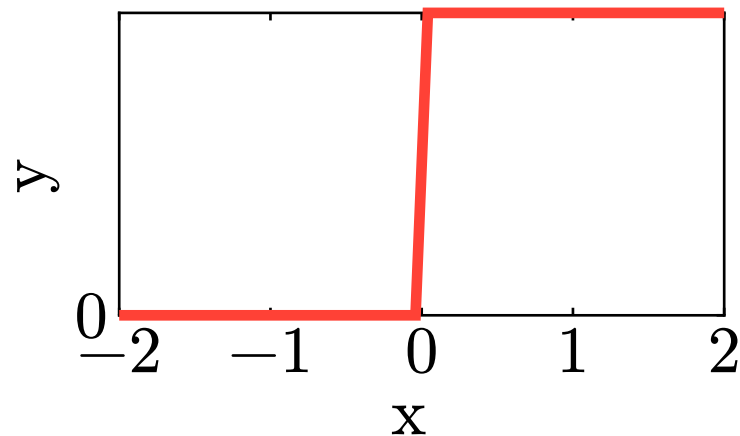
The axon fires only if the voltage is over a threshold

Step function $\sigma(x) = H(x)$



Artificial Neurons

Step function $\sigma(x) = H(x)$



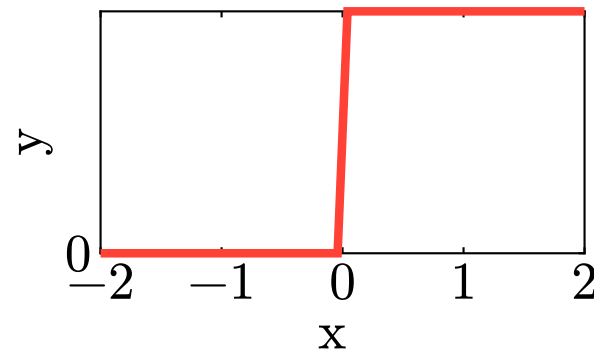
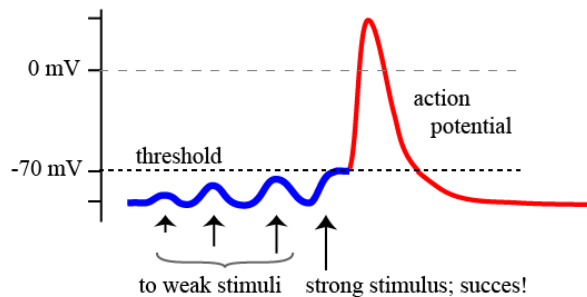
$$f\left(\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}\right) = \sigma\left(\sum_{j=1}^{d_x} \theta_j x_j\right)$$

We call σ the **activation function**

Tells us when the neural fires (activates)

Artificial Neurons

Maybe we want to vary the activation threshold



$$f\left(\begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_{d_x} \end{bmatrix}, \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{d_x} \end{bmatrix}\right) = \sigma\left(\theta_0 + \sum_{j=1}^{d_x} \theta_j x_j\right) = \sigma\left(\sum_{j=0}^{d_x} \theta_j x_j\right)$$

$$\overline{\mathbf{x}} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix}, \quad f(\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \overline{\mathbf{x}})$$

Artificial Neurons

$$f(\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \bar{\mathbf{x}})$$

This is the artificial neuron!

Let us write out the full equation for a neuron

$$f(\mathbf{x}, \boldsymbol{\theta}) = \sigma(\theta_0 1 + \theta_1 x_1 + \dots + \theta_{d_x} x_{d_x})$$

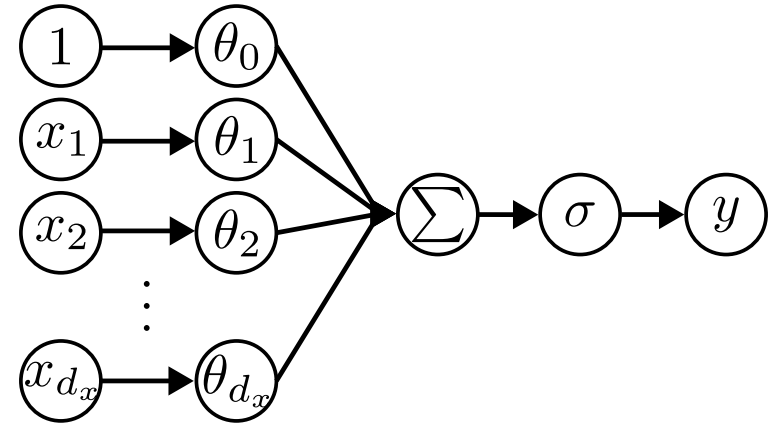
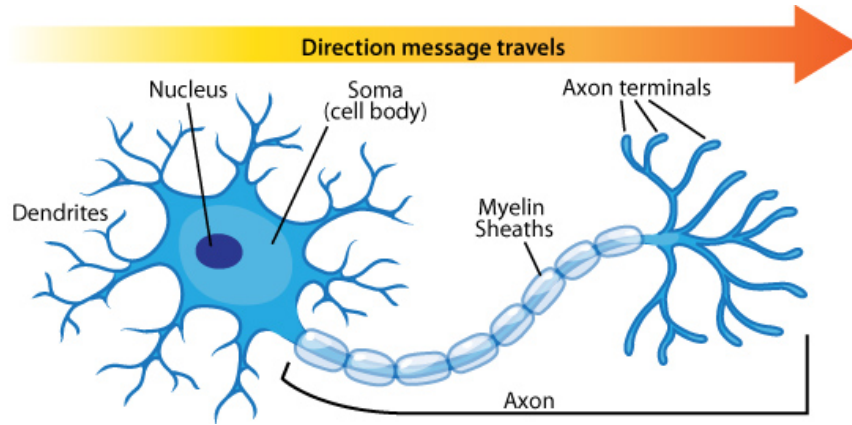
Question: Does this look familiar to anyone?

Answer: Inside σ is the multivariate linear model!

$$f(\mathbf{x}, \boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + \dots + \theta_{d_x} x_{d_x}$$

Artificial Neurons

We model a neuron using a linear model and activation function



$$f(x, \theta) = \sigma(\theta^\top \bar{x})$$

Artificial Neurons

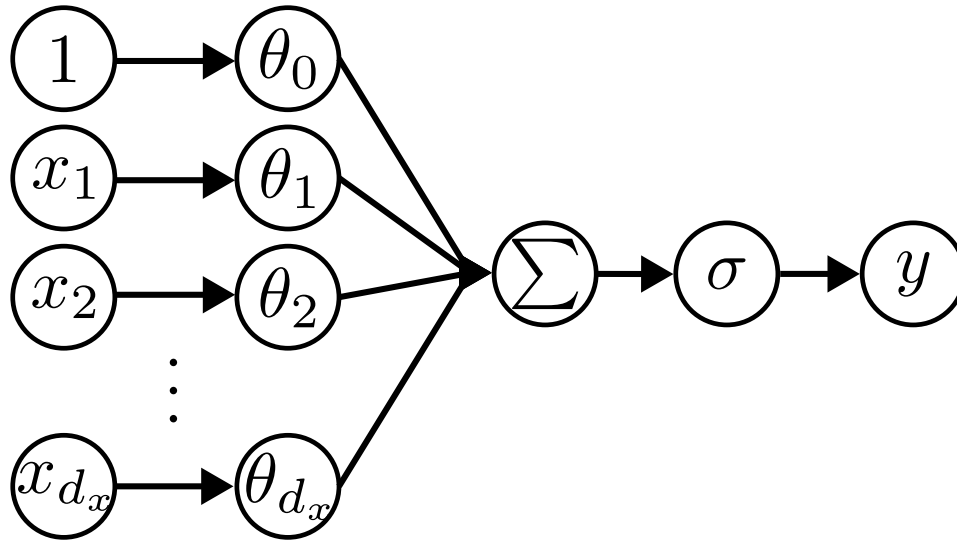
$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \bar{\boldsymbol{x}})$$

Sometimes, we will write $\boldsymbol{\theta}$ as a bias and weight $b, \boldsymbol{w}$

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ \boldsymbol{w} \end{bmatrix}; \quad \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{d_x} \end{bmatrix} = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_{d_x} \end{bmatrix}$$

$$f\left(\boldsymbol{x}, \begin{bmatrix} b \\ \boldsymbol{w} \end{bmatrix}\right) = \sigma(b + \boldsymbol{w}^\top \boldsymbol{x})$$

Artificial Neurons



In machine learning, we represent functions

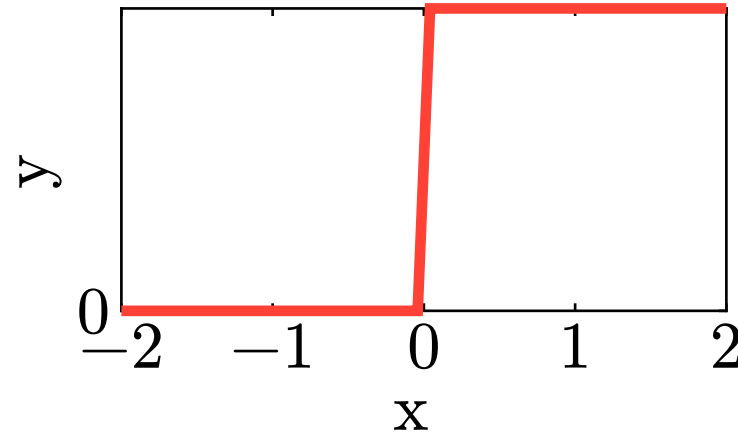
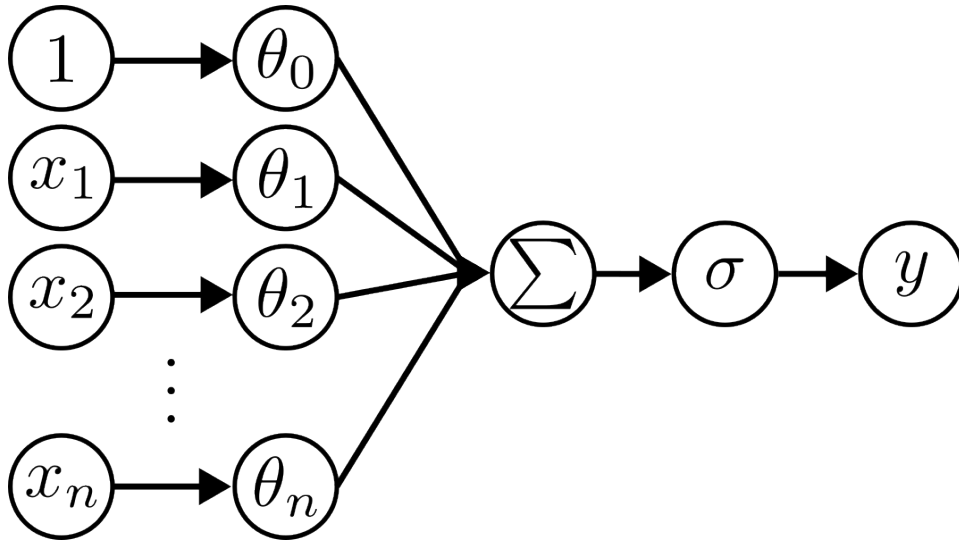
What kinds of functions can our neuron represent?

Let us consider some **boolean** functions

Let us start with a logical AND function

Artificial Neurons

Review: Activation function
(Heaviside step function)



$$\sigma(x) = H(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Artificial Neurons

Implement AND using an artificial neuron

$$f\left([x_1 \ x_2]^\top, [\theta_0 \ \theta_1 \ \theta_2]^\top\right) = \sigma(\theta_0 1 + \theta_1 x_1 + \theta_2 x_2)$$

$$\boldsymbol{\theta} = [\theta_0 \ \theta_1 \ \theta_2]^\top = [-1 \ 1 \ 1]^\top$$

x_1	x_2	y	$f(x_1, x_2, \boldsymbol{\theta})$	$\hat{y}$
0	0	0	$\sigma(-1 \cdot 1 + 1 \cdot 0 + 1 \cdot 0) = \sigma(-1)$	0
0	1	0	$\sigma(-1 \cdot 1 + 1 \cdot 0 + 1 \cdot 1) = \sigma(0)$	0
1	0	0	$\sigma(-1 \cdot 1 + 1 \cdot 1 + 1 \cdot 0) = \sigma(0)$	0
1	1	1	$\sigma(-1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1) = \sigma(1)$	1

Artificial Neurons

Implement OR using an artificial neuron

$$f\left([x_1 \ x_2]^\top, [\theta_0 \ \theta_1 \ \theta_2]^\top\right) = \sigma(\theta_0 1 + \theta_1 x_1 + \theta_2 x_2)$$

$$\boldsymbol{\theta} = [\theta_0 \ \theta_1 \ \theta_2]^\top = [0 \ 1 \ 1]^\top$$

x_1	x_2	y	$f(x_1, x_2, \boldsymbol{\theta})$	$\hat{y}$
0	0	0	$\sigma(1 \cdot 0 + 1 \cdot 0 + 1 \cdot 0) = \sigma(0)$	0
0	1	0	$\sigma(1 \cdot 0 + 1 \cdot 1 + 1 \cdot 0) = \sigma(1)$	1
1	0	1	$\sigma(1 \cdot 0 + 1 \cdot 0 + 1 \cdot 1) = \sigma(1)$	1
1	1	1	$\sigma(1 \cdot 0 + 1 \cdot 1 + 1 \cdot 1) = \sigma(2)$	1

Artificial Neurons

Implement XOR using an artificial neuron

$$f\left([x_1 \ x_2]^\top, [\theta_0 \ \theta_1 \ \theta_2]^\top\right) = \sigma(\theta_0 1 + \theta_1 x_1 + \theta_2 x_2)$$

$$\boldsymbol{\theta} = [\theta_0 \ \theta_1 \ \theta_2]^\top = [? \ ? \ ?]^\top$$

x_1	x_2	y	$f(x_1, x_2, \boldsymbol{\theta})$	$\hat{y}$
0	0	0	This is IMPOSSIBLE!	
0	1	1		
1	0	1		
1	1	0		

Artificial Neurons

Why can't we represent XOR using a neuron?

$$f\left([x_1 \ x_2]^\top, [\theta_0 \ \theta_1 \ \theta_2]^\top\right) = \sigma(1\theta_0 + x_1\theta_1 + x_2\theta_2)$$

We can only represent σ (linear function)

XOR is not a linear combination of x_1, x_2 !

We want to represent any function, not just linear functions

Let us think back to biology, maybe it has an answer

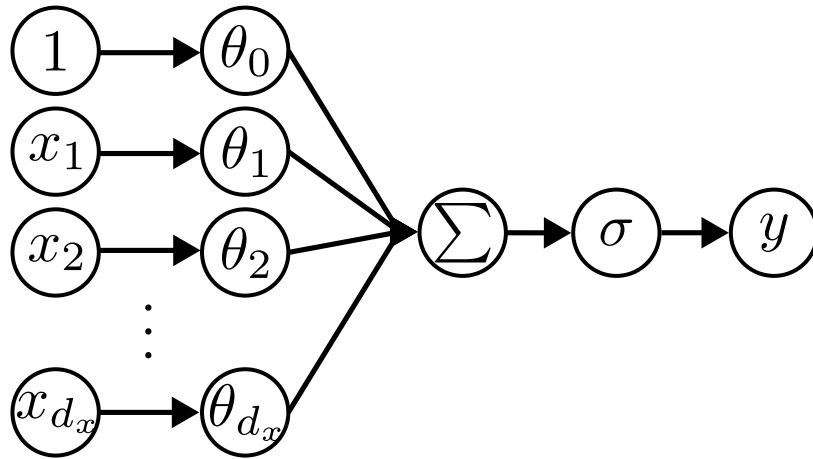
Artificial Neurons

Brain: Biological neurons $\rightarrow$ Biological neural network

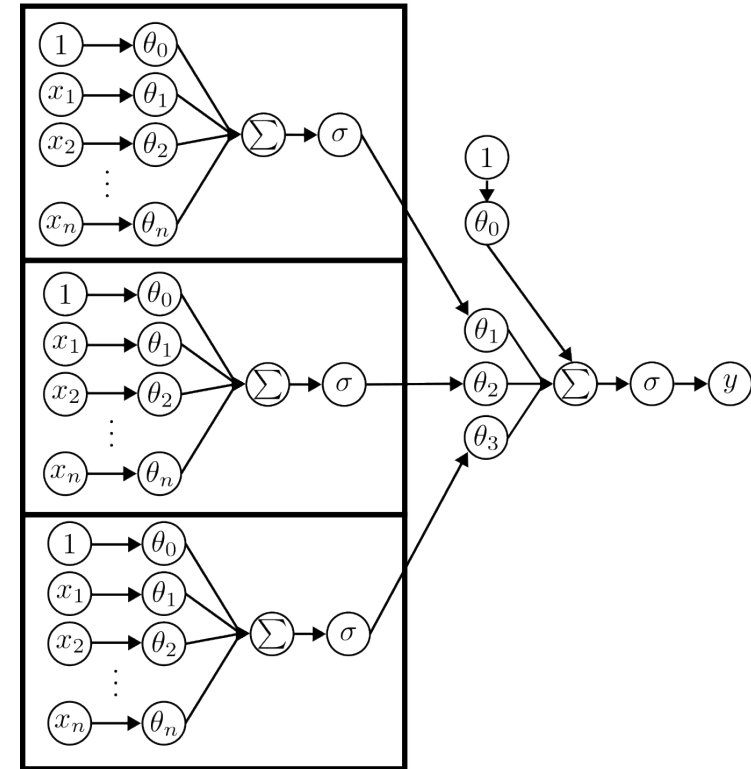
Computer: Artificial neurons $\rightarrow$ Artificial neural network

Artificial Neurons

Connect artificial neurons into a network

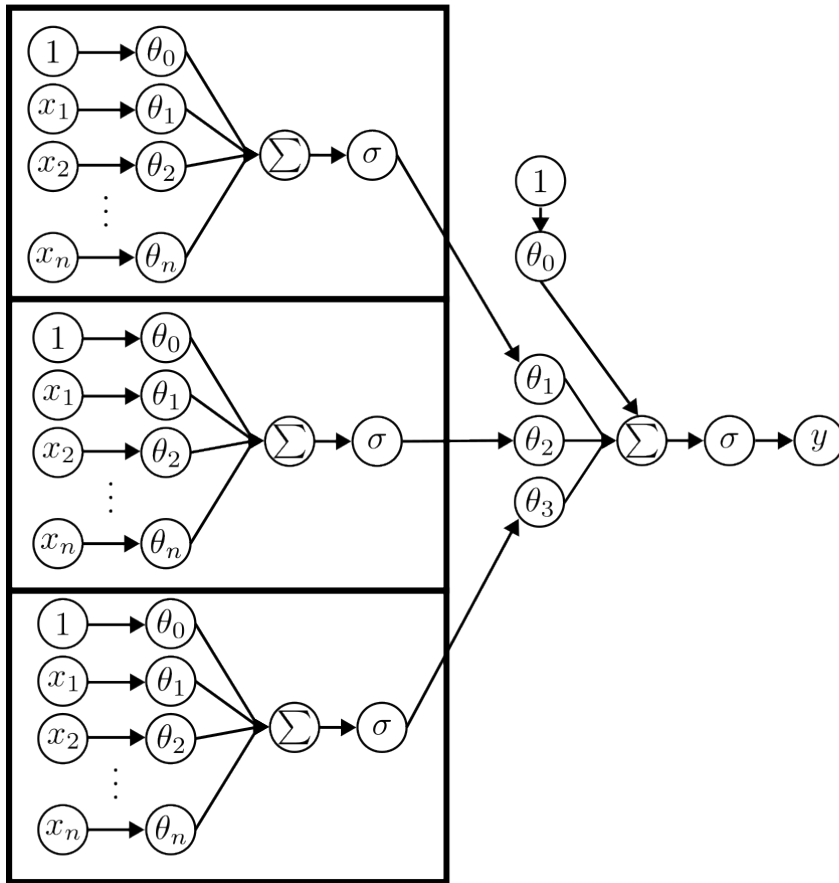


Neuron



Neural Network

Artificial Neurons



Adding neurons in **parallel**
creates a **wide** neural network

Adding neurons in **series** creates a
deep neural network

Today's powerful neural networks
are both **wide** and **deep**

Wide Neural Networks

Wide Neural Networks

How do we express a **wide** neural network mathematically?

A single neuron:

$$f : \mathbb{R}^{d_x} \times \Theta \mapsto \mathbb{R}$$

$$\Theta = \mathbb{R}^{d_x+1}$$

d_y neurons (wide):

$$f : \mathbb{R}^{d_x} \times \Theta \mapsto \mathbb{R}^{d_y}$$

$$\Theta = \mathbb{R}^{(d_x+1) \times d_y}$$

Wide Neural Networks

For a single neuron:

$$f\left(\begin{bmatrix} x_1 \\ \vdots \\ x_{d_x} \end{bmatrix}, \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{d_x} \end{bmatrix}\right) = \sigma\left(\sum_{i=0}^{d_x} \theta_i \bar{x}_i\right)$$

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(b + \boldsymbol{w}^\top \boldsymbol{x})$$

Wide Neural Networks

For a wide network:

$$f\left(\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d_x} \end{bmatrix}, \begin{bmatrix} \theta_{0,1} & \theta_{0,2} & \cdots & \theta_{0,d_y} \\ \theta_{1,1} & \theta_{1,2} & \cdots & \theta_{1,d_y} \\ \vdots & \vdots & \ddots & \vdots \\ \theta_{d_x,1} & \theta_{d_x,2} & \cdots & \theta_{d_x,d_y} \end{bmatrix}\right) = \begin{bmatrix} \sigma\left(\sum_{i=0}^{d_x} \theta_{i,1} \bar{x}_i\right) \\ \sigma\left(\sum_{i=0}^{d_x} \theta_{i,2} \bar{x}_i\right) \\ \vdots \\ \sigma\left(\sum_{i=0}^{d_x} \theta_{i,d_y} \bar{x}_i\right) \end{bmatrix}$$

$$f(\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \bar{\mathbf{x}}); \quad \boldsymbol{\theta}^\top \in \mathbb{R}^{d_y \times (d_x + 1)}$$

$$f\left(\mathbf{x}, \begin{bmatrix} \mathbf{b} \\ \mathbf{W} \end{bmatrix}\right) = \sigma(\mathbf{b} + \mathbf{W}^\top \mathbf{x}); \quad \mathbf{b} \in \mathbb{R}^{d_y}, \mathbf{W} \in \mathbb{R}^{d_x \times d_y}$$

Deep Neural Networks

Deep Neural Networks

How do we express a **deep** neural network mathematically?

A wide network and deep network have a similar function signature:

$$f : \mathbb{R}^{d_x} \times \Theta \mapsto \mathbb{R}^{d_y}$$

But the parameters change!

Wide: $\Theta = \mathbb{R}^{(d_x+1) \times d_y}$

Deep: $\Theta = \mathbb{R}^{(d_x+1) \times d_h} \times \mathbb{R}^{(d_h+1) \times d_h} \times \dots \times \mathbb{R}^{(d_h+1) \times d_y}$

$$\boldsymbol{\theta} = [\boldsymbol{\theta}_1 \ \boldsymbol{\theta}_2 \ \dots \ \boldsymbol{\theta}_\ell]^\top = [\boldsymbol{\varphi} \ \boldsymbol{\psi} \ \dots \ \boldsymbol{\xi}]^\top$$

Deep Neural Networks

A wide network:

$$f(x, \theta) = \sigma(\theta^\top \bar{x})$$

A deep network has many internal functions

$$f_1(x, \varphi) = \sigma(\varphi^\top \bar{x}) \quad f_2(x, \psi) = \sigma(\psi^\top \bar{x}) \quad \dots \quad f_\ell(x, \xi) = \sigma(\xi^\top \bar{x})$$

$$f(x, \theta) = f_\ell(\dots f_2(f_1(x, \varphi), \psi) \dots \xi)$$

Deep Neural Networks

Written another way

$$z_1 = f_1(x, \varphi) = \sigma(\varphi^\top \bar{x})$$

$$z_2 = f_2(z_1, \psi) = \sigma(\psi^\top \bar{z}_1)$$

$\vdots$

$$\hat{y} = f_\ell(x, \xi) = \sigma(\xi^\top \bar{z}_{\ell-1})$$

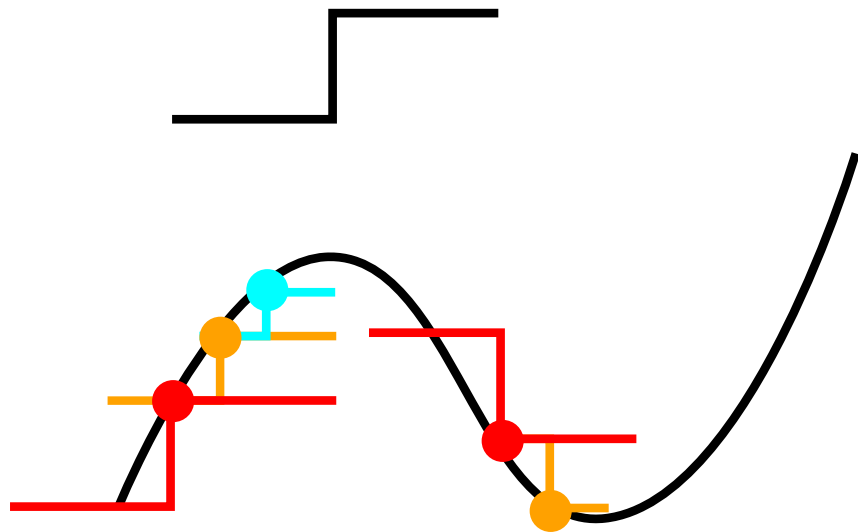
We call each function a **layer**

A deep neural network is made of many layers

Deep Neural Networks

What functions can we represent using a deep neural network?

Proof Sketch: Approximate a continuous function $g : \mathbb{R} \mapsto \mathbb{R}$ using a linear combination of Heaviside functions



$$\exists (\boldsymbol{\theta} \in \mathbb{R}^{1 \times d_h}, \boldsymbol{\varphi} \in \mathbb{R}^{(d_h+1) \times d_1}) \text{ such that } \lim_{d_h \mapsto \infty} [\boldsymbol{\varphi}^\top \sigma(\overline{\boldsymbol{\theta}^\top \overline{x}})]$$

Deep Neural Networks

A deep neural network is a **universal function approximator**

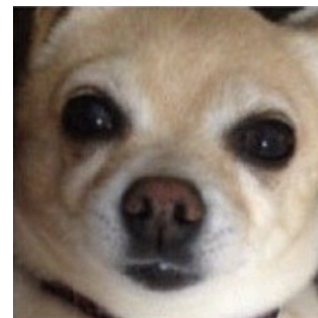
It can approximate **any** continuous function $g(x)$ to precision ε

$$| g(x) - f(x, \theta) | < \varepsilon$$

Making the network deeper or wider decreases ε

Very powerful finding! The basis of deep learning.

Task: predict how many ❤️ a photo gets on social media

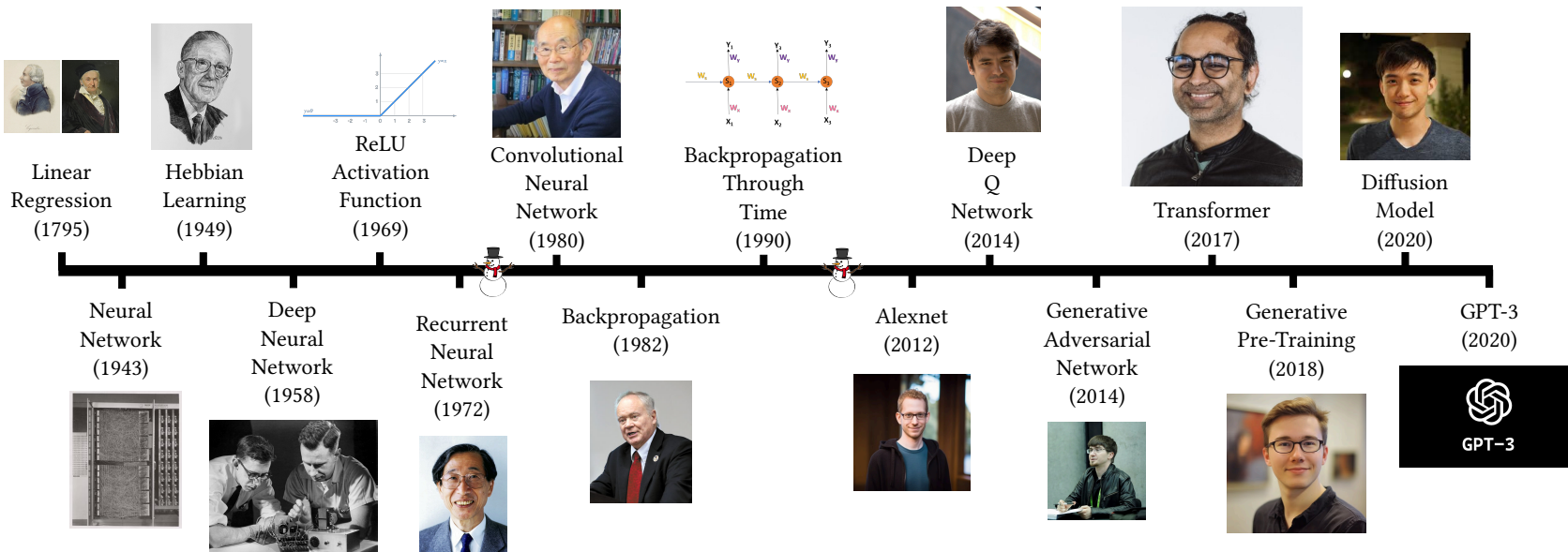


Practical Considerations

Practical Considerations

We call wide neural networks **perceptrons**

We call deep neural networks **multi-layer perceptrons** (MLP)



Practical Considerations

All the models we examine in this course will use MLPs

- Recurrent neural networks
- Graph neural networks
- Transformers
- Generative image models
- Chatbots

It is very important to understand MLPs!

I will explain them again very simply

Practical Considerations

A **layer** is a linear operation and an activation function

$$f(\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \overline{\mathbf{x}})$$

$$f\left(\mathbf{x}, \begin{bmatrix} \mathbf{b} \\ \mathbf{W} \end{bmatrix}\right) = \sigma(\mathbf{b} + \mathbf{W}^\top \mathbf{x})$$

Many layers makes a deep neural
network

$$z_1 = f(\mathbf{x}, \boldsymbol{\theta})$$

$$z_2 = f(z_1, \boldsymbol{\theta})$$

$$\hat{y} = f(z_2, \boldsymbol{\theta})$$

Practical Considerations

Let us create a wide neural network in colab! <https://colab.research.google.com/drive/1uCLnERcamA4PX-ozWZlGivq3m8UEJwiW?usp=sharing>

Practical Considerations

Linear regression:

- + Analytical solution
- + Low data requirement
- Poor scalability
- Poor polynomials generalization

Neural networks:

- No analytical solution
- Big data requirement
- + Scale to large inputs
- + Slightly better generalization

Next time, we will find out how to train our neural network

Unlike linear regression, finding θ is much more difficult for neural networks